01_Fundamentals_of_Artificial_Intelligence_and_Python.ipynb

Instructor: Priyam Chakraborty, PhD (Chanakya Postdoctoral Fellow at Dept of AI & AI4ICPS, IIT Kharagpur)
Audience: Anyone with an interest in AI/ML — some Python familiarity assumed (variables, loops, functions).
Duration: 2 hours hands-on.

Learning objectives (by the end of this notebook)

1. Set up a reproducible Python environment for ML (local / Colab).
2. Use NumPy for vectorized numerical computation and understand broadcasting.
3. Use pandas for quick tabular exploration (we will expand on this in Chunk 2).
4. Visualize data (matplotlib) and display images from datasets.
5. Implement a small supervised learning pipeline using scikit-learn (we'll cover a compact demo here).
6. Understand the difference between NumPy arrays and PyTorch tensors (preview), and extract embeddings from a pretrained vision model (bridge to Notebook 2).

How this notebook fits the course

- This notebook builds the foundation required for Notebook 2 (Surrogates & ViT).
- We emphasize clarity: shapes, types, and dataflow (data → model → predictions → evaluation).

Practical notes (read before running)

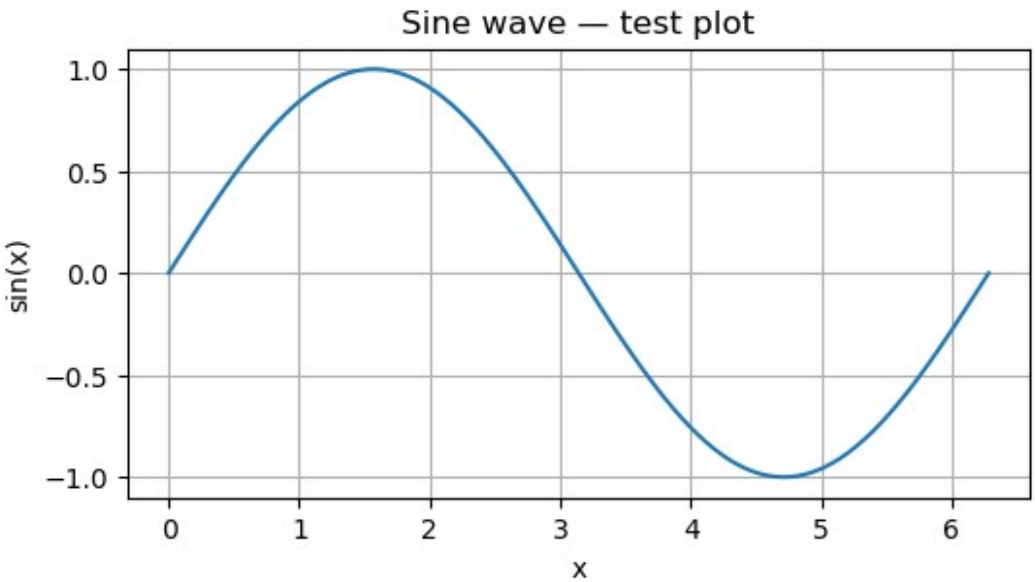
- **Colab vs local:**
  - If using Google Colab: Runtime → Change runtime type → GPU recommended for later notebooks (not strictly needed here).
  - If local: ensure a recent Python (3.9+) and pip; using a virtualenv / conda env is recommended.
- **Expected packages:** `numpy`, `pandas`, `matplotlib`, `scikit-learn`, `torch`, `torchvision`.

- **Interactive tips:** run cells sequentially. If something errors, read the error and inspect shapes / dtypes — that's a common learning moment.
- **Reproducibility:** We set seeds where relevant. Exact bitwise reproducibility on GPU may still vary across systems — that's OK for class demos.

```
import sys, os
print("Python executable:", sys.executable)
print("Python version:", sys.version.split()[0])
print("Working dir:", os.getcwd())

# Quick sanity prints
print("Numpy version:", __import__('numpy').__version__)
print("Pandas version:", __import__('pandas').__version__)
print("Matplotlib version:", __import__('matplotlib').__version__)
print("Torch available:", __import__('torch').__version__ if 'torch'
in sys.modules or True else 'not installed')
```

```
Python executable: N:\Software\python.exe
Python version: 3.12.7
Working dir: C:\Users\hp prodesk
Numpy version: 1.26.4
Pandas version: 2.2.2
Matplotlib version: 3.9.2
Torch available: 2.8.0+cpu
```

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)

plt.figure(figsize=(6,3))
plt.plot(x, y)
plt.title("Sine wave — test plot")
plt.xlabel("x")
plt.ylabel("sin(x)")
plt.grid(True)
plt.show()
```




```
# If you're on Colab, uncomment the following line to install/update
packages.
# !pip install --quiet torch torchvision scikit-learn pandas
matplotlib

# Standard imports for this course
import os
import math
import random
from typing import Tuple, List, Dict

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Scikit-learn utilities (we'll use them in Chunk 3 as well)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    roc_auc_score, roc_curve, precision_recall_curve
)

# Torch import (we'll use it in later chunks; import now for
convenience)
import torch
import torchvision

# Display settings for notebooks
%matplotlib inline
plt.rcParams['figure.figsize'] = (8, 5)
```

```
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'
plt.show()

# Reproducibility helpers
def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    # If CUDA available, set cuda seed too
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

# Device selection
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {DEVICE}")
```

```
Using device: cpu
```

NumPy refresher — vectors, broadcasting, and performance

NumPy is the backbone of numerical computing in Python:

- Implements **ndarrays**: contiguous, typed, N-dimensional arrays (fast).
- Provides vectorized operations (no Python loops) which are **much faster** than naive Python loops.
- **Broadcasting** lets you operate on arrays with different shapes in a consistent way (we'll demonstrate).

We'll do a few short, focused exercises:

1. Create arrays and practice broadcasting.
2. Implement mean and variance manually (vectorized) and compare to `np.mean` / `np.var`.

```
# Create 1D and 2D arrays
a = np.array([1.0, 2.0, 3.0])
b = np.array([[1, 2, 3],
              [4, 5, 6]], dtype=float)

print("a:", a, "shape:", a.shape)
print("b:\n", b, "shape:", b.shape)

# Broadcasting example: add vector a to each row of b
c = b + a  # shapes (2,3) + (3,) => broadcasting to (2,3)
print("\nBroadcasted b + a =>\n", c)
```

```python
# Elementwise ops and reductions
print("\nReductions: sum(a) =", a.sum(), " mean(b) =", b.mean())
print("Dot product a·a:", a.dot(a))
```

```
a: [1. 2. 3.] shape: (3,)
b:
 [[1. 2. 3.]
 [4. 5. 6.]] shape: (2, 3)

Broadcasted b + a =>
 [[2. 4. 6.]
 [5. 7. 9.]]

Reductions: sum(a) = 6.0  mean(b) = 3.5
Dot product a·a: 14.0
```

Exercise 1 (hands-on)

Implement functions `my_mean` and `my_var` that compute mean and variance along a specified axis (vectorized, no Python loops). Compare results with `np.mean` and `np.var`.

Hints:

- Use `np.sum` and divide by the number of elements.
- For variance, use the definition `E[x^2] - (E[x])^2` (numerically stable alternatives exist but this is fine for the exercise).

```python
def my_mean(x: np.ndarray, axis=None):
    """
    Compute mean along axis (None for global mean).
    """
    # number of elements along axis
    if axis is None:
        n = x.size
    else:
        n = x.shape[axis]
    return np.sum(x, axis=axis) / n

def my_var(x: np.ndarray, axis=None):
    """
    Compute variance using E[x^2] - E[x]^2
    """
    ex = my_mean(x, axis=axis)
    ex2 = my_mean(x**2, axis=axis)
    return ex2 - ex**2

# Quick tests
X = np.random.randn(1000, 10)
print("my_mean vs np.mean:", np.allclose(my_mean(X, axis=0),
```

```
np.mean(X, axis=0)))
print("my_var vs np.var  :", np.allclose(my_var(X, axis=0), np.var(X,
axis=0)))
```

```
my_mean vs np.mean: True
my_var vs np.var  : True
```

Broadcasting notes & pitfalls

- Broadcasting is powerful but can *silently* change shapes. Always inspect shapes when results are surprising.
- Example pitfalls:
  - `(n,1)` vs `(n,)` — these broadcast differently with other arrays; prefer explicit reshapes: `x.reshape(-1,1)`.
  - Watch out for integer division in older Python/NumPy versions (not an issue typically now).
- When in doubt: print `.shape` of arrays before operations.

Quick checkpoint (conceptual, 2 min)

1. Why is NumPy faster than Python loops for numerical ops?
2. What does broadcasting allow you to do?
3. In the variance formula `E[x^2] - E[x]^2`, why might numerical stability be a concern for large numbers?

(Answers in a later cell — try to answer on your own first.)

```
answers = {
    1: "NumPy uses contiguous C-backed arrays and vectorized C
implementations, avoiding Python-level loops and overhead.",
    2: "Broadcasting allows arithmetic between arrays of different
shapes by 'stretching' dimensions with size 1 to match the other
array.",
    3: "When numbers are large, subtracting two large similar numbers
(E[x^2] and E[x]^2) can lead to catastrophic cancellation and loss of
precision. Numerically stable formulas (e.g., Welford's algorithm) are
preferred for streaming/large data."
}
answers
```

```
{1: 'NumPy uses contiguous C-backed arrays and vectorized C
implementations, avoiding Python-level loops and overhead.',
 2: "Broadcasting allows arithmetic between arrays of different shapes
by 'stretching' dimensions with size 1 to match the other array.",
 3: "When numbers are large, subtracting two large similar numbers
(E[x^2] and E[x]^2) can lead to catastrophic cancellation and loss of
precision. Numerically stable formulas (e.g., Welford's algorithm) are
preferred for streaming/large data."}
```

End of Chunk 1.

What we covered:

- Notebook setup, package imports, reproducibility controls.
- NumPy refresh: arrays, broadcasting, reductions, simple exercises.

Next (Chunk 2) we'll dive into **pandas for tabular EDA** and **visualization**, and then quickly show how to display images from torchvision datasets (Fashion-MNIST) so you get hands-on experience with image arrays.

Chunk 2 — pandas for Exploratory Data Analysis (EDA) & Visualizing Images

Goals for this chunk:

- Practice pandas for tabular exploration and basic cleaning (missing values, groupby, aggregates).
- Learn how to visualize distributions and relationships with `matplotlib`.
- Load a small image dataset (Fashion-MNIST) and view images inside the notebook.
- Use PCA to get a 2D visualization of high-dimensional image data (intuition for embeddings).

Notes:

- We keep the datasets small so notebooks remain interactive on Colab or modest local machines.
- We will convert image datasets into lightweight pandas structures (metadata + simple features) for demonstrations.

```
# On Colab, you may need torchvision; uncomment if necessary:
# !pip install --quiet torchvision

import torchvision
from torchvision import transforms, datasets
from torch.utils.data import DataLoader, Subset

# Helper: show a grid of images (expects 2D numpy arrays or torch
tensors)
def show_image_grid(images, titles=None, ncols=5, figsize=(12,6)):
    """
    images: list of numpy arrays or torch tensors shaped (H,W) or
(H,W,C)
    titles: optional list of titles
    """
    n = len(images)
    ncols = min(ncols, n)
    nrows = math.ceil(n / ncols)
    plt.figure(figsize=figsize)
    for i, img in enumerate(images):
        plt.subplot(nrows, ncols, i+1)
        if isinstance(img, torch.Tensor):
```

```
            arr = img.detach().cpu().numpy()
        else:
            arr = img
        # If grayscale with shape (H,W) or (1,H,W)
        if arr.ndim == 3 and arr.shape[0] == 1:
            arr = arr.squeeze(0)
        if arr.ndim == 3 and arr.shape[2] == 1:
            arr = arr.squeeze(2)
        plt.imshow(arr, cmap='gray')
        plt.axis('off')
        if titles is not None:
            plt.title(str(titles[i]), fontsize=10)
    plt.tight_layout()
    plt.show()
```

Load Fashion-MNIST

Fashion-MNIST is a small image dataset (28×28 grayscale) of clothing items with 10 classes. It's great for teaching because it's lightweight and visually interpretable.

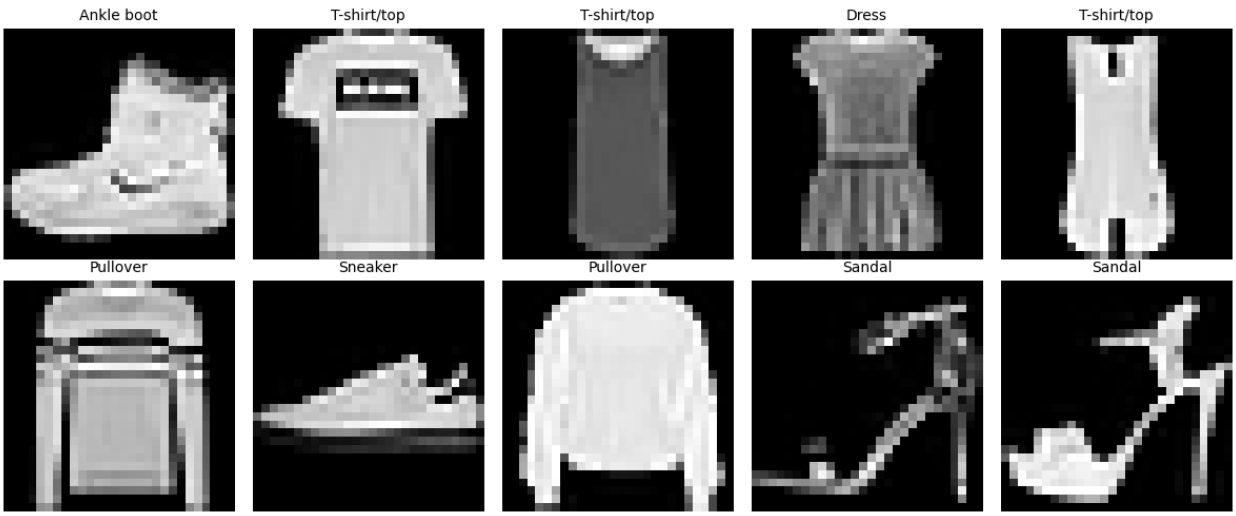
We'll:

1. Load the train split via `torchvision.datasets`.
2. Inspect shapes and a few examples.
3. Build a small pandas DataFrame holding metadata (index, label, mean intensity).

```
# Setup transform to return tensors in [0,1]
transform = transforms.Compose([transforms.ToTensor()])

# Download train split (stored in ~/.cache or ./data)
data_dir = "./data"
os.makedirs(data_dir, exist_ok=True)
train_ds = datasets.FashionMNIST(root=data_dir, train=True,
download=True, transform=transform)

print("Dataset size (train):", len(train_ds))
sample_img, sample_label = train_ds[0]
print("Sample image tensor shape:", sample_img.shape, " label:",
sample_label)

# Show some sample images and labels
labels_map = {
    0: "T-shirt/top", 1: "Trouser", 2: "Pullover", 3: "Dress", 4:
"Coat",
    5: "Sandal", 6: "Shirt", 7: "Sneaker", 8: "Bag", 9: "Ankle boot"
}

# Display first 10 images
imgs = [train_ds[i][0] for i in range(10)]
```

```
titles = [labels_map[int(train_ds[i][1])] for i in range(10)]
show_image_grid(imgs, titles=titles, ncols=5, figsize=(12,5))
```

```
100%|██████████| 26.4M/26.4M [00:02<00:00, 10.6MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 159kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 3.27MB/s]
100%|██████████| 5.15k/5.15k [00:00<?, ?B/s]

Dataset size (train): 60000
Sample image tensor shape: torch.Size([1, 28, 28])  label: 9
```



Create a lightweight pandas DataFrame

We'll create a DataFrame with:

- `idx` — dataset index
- `label` — integer label
- `label_name` — human-readable label
- `mean_intensity` — mean pixel value of the image (simple feature)
- `std_intensity` — std of pixel values

This is useful to demonstrate pivoting, groupby, and handling missing values.

```
# Build a small DataFrame for the first N samples (keep N small for
speed)
N = 5000  # choose 5000 (half the dataset) to keep operations fast in
notebooks
idxs = list(range(N))

meta = []
for i in idxs:
    img_tensor, lbl = train_ds[i]
    # img_tensor shape (1,28,28) -> convert to numpy 2D
```

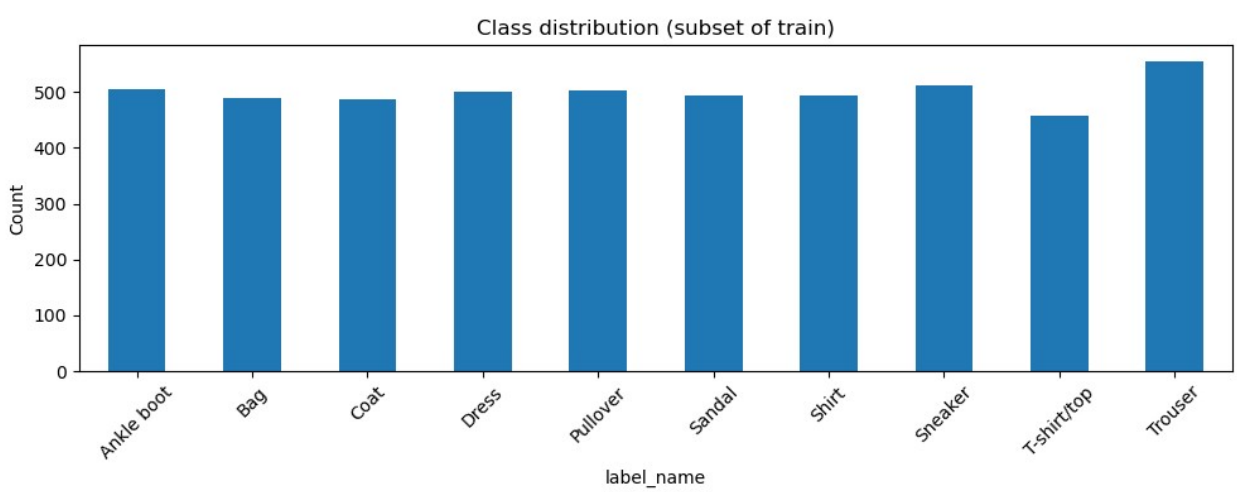
```
    arr = img_tensor.squeeze(0).numpy()
    mean_int = float(arr.mean())
    std_int = float(arr.std())
    meta.append((i, int(lbl), labels_map[int(lbl)], mean_int,
std_int))

df = pd.DataFrame(meta, columns=["idx", "label", "label_name",
"mean_intensity", "std_intensity"])
df.head()
```

```
   idx  label   label_name  mean_intensity  std_intensity
0    0      9   Ankle boot        0.381388       0.399186
1    1      0  T-shirt/top        0.423159       0.395417
2    2      0  T-shirt/top        0.143367       0.194897
3    3      3        Dress        0.233338       0.254311
4    4      0  T-shirt/top        0.306057       0.407228
```

```
# Basic statistics
print(df.describe(include='all'))

# Class distribution
class_counts = df['label_name'].value_counts().sort_index()
print("\nClass counts (first 10 shown):\n", class_counts)

# Plot class distribution
plt.figure(figsize=(10,4))
class_counts.plot(kind='bar')
plt.title("Class distribution (subset of train)")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Introduce some missing values artificially (exercise later)
df_missing = df.copy()
# randomly set 1% of mean_intensity to NaN
mask = np.random.choice(df_missing.index,
size=int(0.01*len(df_missing)), replace=False)
df_missing.loc[mask, "mean_intensity"] = np.nan

print("Missing values per column:\n", df_missing.isnull().sum())
```

```
                idx        label label_name  mean_intensity
std_intensity
count   5000.000000  5000.000000       5000     5000.000000
5000.000000
unique          NaN          NaN         10             NaN
NaN
top             NaN          NaN    Trouser             NaN
NaN
```

```
freq            NaN          NaN         556              NaN
NaN
mean    2499.500000     4.500000         NaN         0.286146
0.322145
std     1443.520003     2.868945         NaN         0.125473
0.077894
min        0.000000     0.000000         NaN         0.033874
0.074934
25%     1249.750000     2.000000         NaN         0.186601
0.264486
50%     2499.500000     4.000000         NaN         0.271589
0.334012
75%     3749.250000     7.000000         NaN         0.381313
0.385190
max     4999.000000     9.000000         NaN         0.708188
0.475632

Class counts (first 10 shown):
 label_name
Ankle boot     506
Bag            490
Coat           488
Dress          501
Pullover       504
Sandal         493
Shirt          493
Sneaker        512
T-shirt/top    457
Trouser        556
Name: count, dtype: int64
```




```
Missing values per column:
 idx                 0
label               0
```

```
label_name         0
mean_intensity    50
std_intensity      0
dtype: int64
```

Exercise 2 (hands-on): Missing data handling

Using `df_missing`:

1. Report rows with missing `mean_intensity`.
2. Create two cleaned versions:
   - `df_drop`: dropping rows with missing values.
   - `df_fill`: filling missing `mean_intensity` with the column median.
3. Compare the class distribution in the cleaned DataFrames to the original `df`. Any class disproportionately affected?

Hint: use `df_missing.isnull()`, `dropna()`, and `fillna()`. Use `groupby('label_name').size()` for comparisons.

```python
# 1. rows with missing mean_intensity
missing_rows = df_missing[df_missing['mean_intensity'].isnull()]
print("Sample missing rows:\n", missing_rows.head())

# 2a. drop rows
df_drop =
df_missing.dropna(subset=['mean_intensity']).reset_index(drop=True)

# 2b. fill with median
median_val = df_missing['mean_intensity'].median()
df_fill = df_missing.copy()
df_fill['mean_intensity'] =
df_fill['mean_intensity'].fillna(median_val)

# Compare class distribution
orig_counts = df['label_name'].value_counts().sort_index()
drop_counts = df_drop['label_name'].value_counts().sort_index()
fill_counts = df_fill['label_name'].value_counts().sort_index()

comparison = pd.DataFrame({
    "orig": orig_counts,
    "drop": drop_counts,
    "fill": fill_counts
}).fillna(0).astype(int)
comparison
```

```
Sample missing rows:
      idx  label  label_name  mean_intensity  std_intensity
21    21      1     Trouser             NaN       0.350733
235  235      6       Shirt             NaN       0.272268
```

```
349  349      5      Sandal              NaN       0.167520
373  373      7     Sneaker              NaN       0.292605
563  563      9  Ankle boot              NaN       0.348426

              orig  drop  fill
label_name
Ankle boot     506   499   506
Bag            490   484   490
Coat           488   486   488
Dress          501   498   501
Pullover       504   499   504
Sandal         493   489   493
Shirt          493   487   493
Sneaker        512   504   512
T-shirt/top    457   452   457
Trouser        556   552   556
```

Visualizing relationships & dimensionality reduction (PCA)

Images are high-dimensional. To get an intuition for clusters/separability, we:

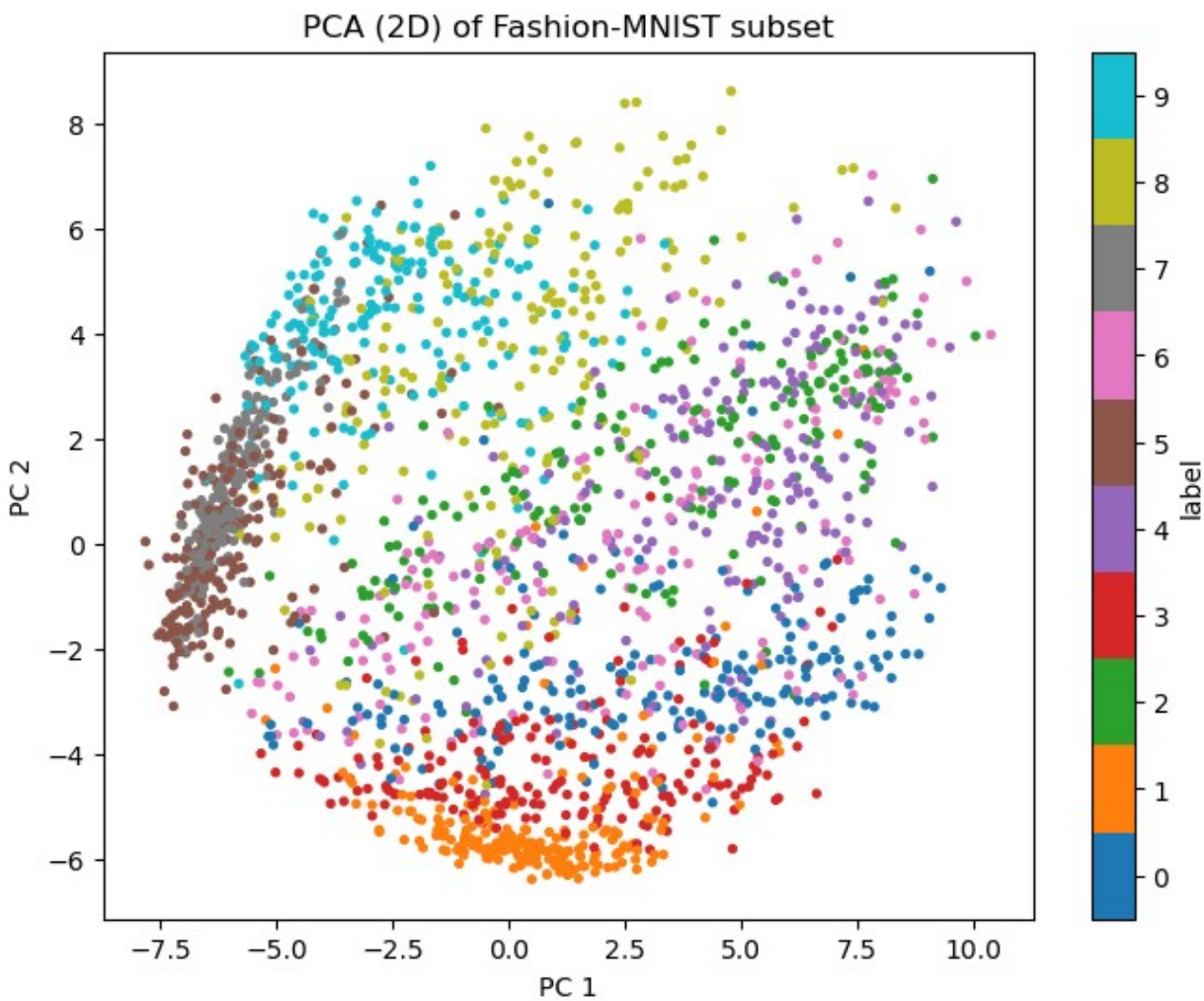
1. Flatten images into vectors (28×28 = 784 dims).
2. Run PCA to reduce to 2D.
3. Plot a scatter of the first two PCs colored by label.

Notes:

- PCA is linear and won't capture complex structure, but it's fast and useful for visualization.
- We will use a random subset (e.g., 2000 samples) to keep the demo responsive.

```
from sklearn.decomposition import PCA

# Select subset
subset_size = 2000
subset_idxs = np.random.choice(df['idx'].values, size=subset_size,
replace=False)

# Build matrix X (subset_size, 784)
X = np.zeros((subset_size, 28*28), dtype=np.float32)
y = np.zeros(subset_size, dtype=int)

for j, idx in enumerate(subset_idxs):
    img_tensor, lbl = train_ds[int(idx)]
    X[j, :] = img_tensor.numpy().reshape(-1)
    y[j] = int(lbl)

# Standardize features before PCA (center)
X_meaned = X - X.mean(axis=0, keepdims=True)
```

```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_meaned)

# Scatter plot
plt.figure(figsize=(8,6))
scatter = plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='tab10', s=8)
plt.title("PCA (2D) of Fashion-MNIST subset")
plt.xlabel("PC 1")
plt.ylabel("PC 2")
plt.colorbar(scatter, ticks=range(10), label='label')
plt.clim(-0.5, 9.5)
plt.show()

print("Explained variance ratio (PC1, PC2):",
pca.explained_variance_ratio_.round(4))
```




```
Explained variance ratio (PC1, PC2): [0.2956 0.1791]
```

Interpretation tips

- Look for clusters and how they align with class labels.
- Some classes (e.g., "Trouser") may cluster separately, while others (e.g., "Shirt" vs "T-shirt") may overlap — reflecting visual similarity.
- PCA captures global linear directions of maximum variance; separability in PCA space suggests easier classification, but overlap does not mean the data is impossible to classify (non-linear features help).

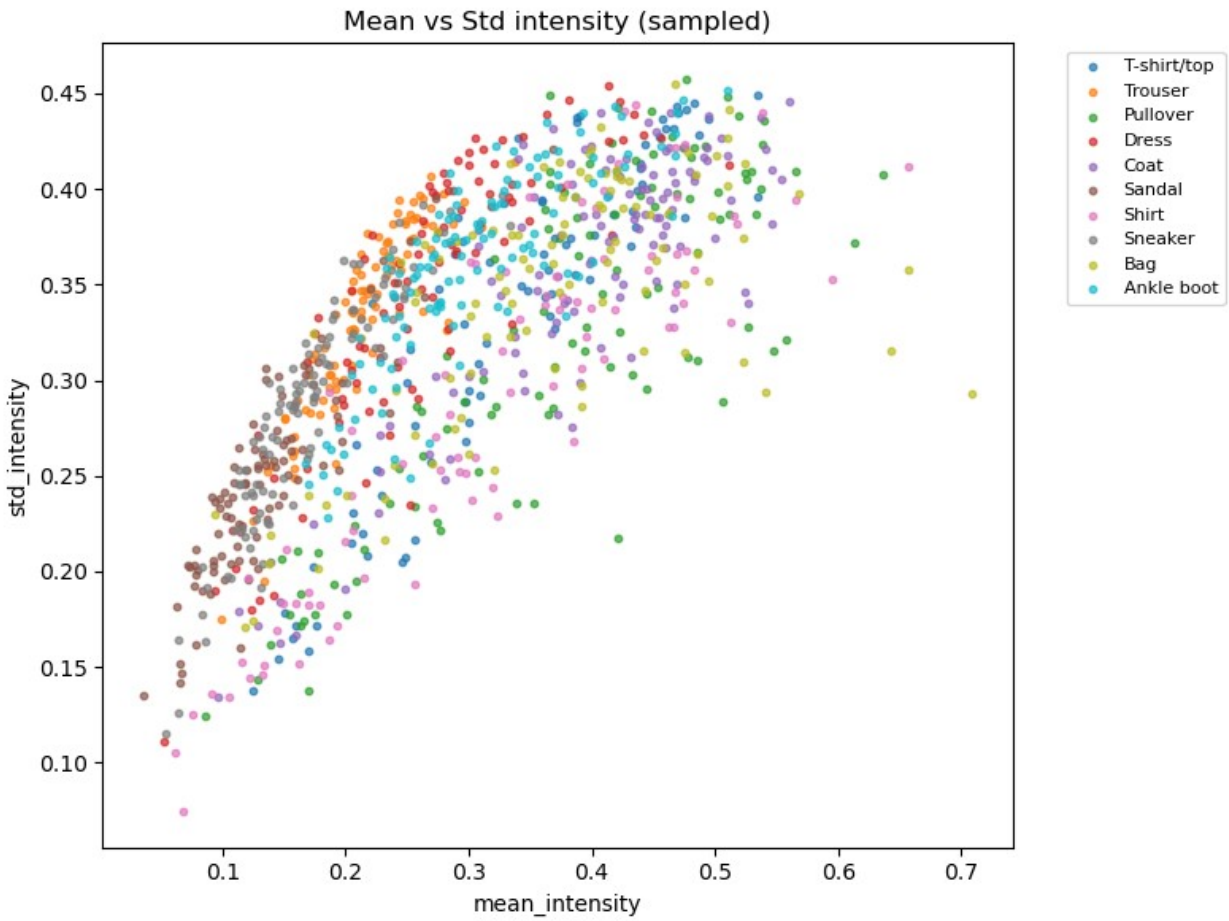
This exercise prepares intuition for embeddings: later, pretrained models produce embeddings that often cluster by semantic content — we will exploit that in Notebook 2.

Quick feature engineering example

We built `mean_intensity` and `std_intensity` as simple per-image features. Simple engineered features can sometimes help small models or serve as sanity checks.

Let's visualize mean vs std by class to see if these features give any separation.

```
# scatter using the DataFrame (subset for speed)
viz_df = df.sample(1000, random_state=42)
plt.figure(figsize=(8,6))
for lbl in sorted(viz_df['label'].unique()):
    part = viz_df[viz_df['label'] == lbl]
    plt.scatter(part['mean_intensity'], part['std_intensity'], s=10,
label=labels_map[int(lbl)], alpha=0.7)
plt.xlabel("mean_intensity")
plt.ylabel("std_intensity")
plt.title("Mean vs Std intensity (sampled)")
plt.legend(bbox_to_anchor=(1.05,1), loc='upper left', fontsize=8)
plt.tight_layout()
plt.show()
```



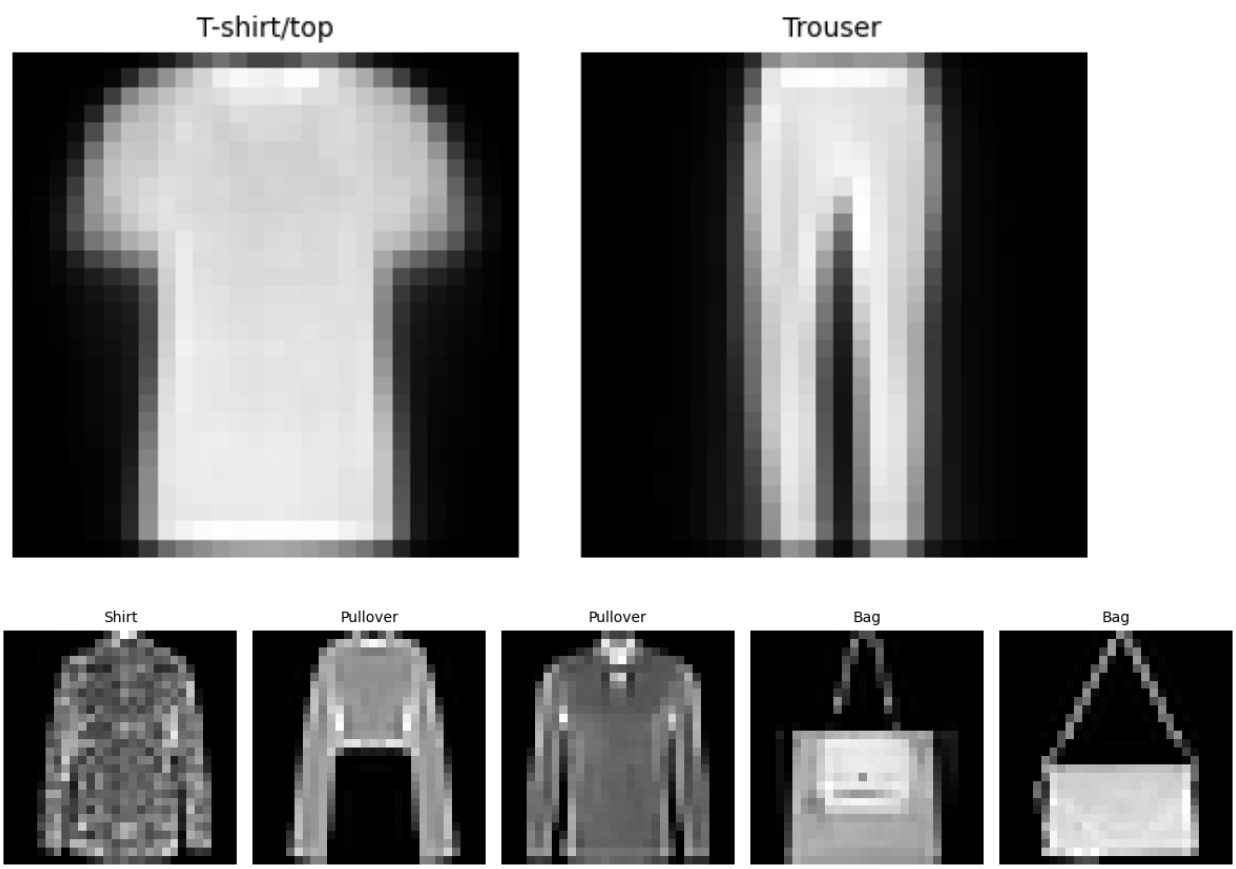
Exercise 3 (hands-on): Visual analysis tasks

1. From `df`, compute the per-class average image (i.e., mean pixel value at each pixel position across all images of a class). Visualize the mean image for two chosen classes — what do you observe?
2. Using the PCA result (`X_pca` and `y`), pick several points that lie in overlapping regions — display the corresponding original images. Do they look ambiguous?
3. Create a scatter plot of `mean_intensity` vs `std_intensity` for just two classes (your choice) and comment whether these two features might help a simple classifier.

Hints: for (1), you will need to sum images per class and divide by count. Remember to reshape flattened arrays back to 28×28 to visualize.

```
# Hint for (1): compute mean image for a class (example for class 0
and 1)
def mean_image_for_label(label, limit=None):
    idxs_label = [i for i in range(N) if df.loc[i, 'label'] == label]
    if limit is not None:
        idxs_label = idxs_label[:limit]
    arrs = [train_ds[int(i)][0].squeeze(0).numpy() for i in
idxs_label]
    return np.mean(np.stack(arrs, axis=0), axis=0)
```

```
mean_img_0 = mean_image_for_label(0)
mean_img_1 = mean_image_for_label(1)
show_image_grid([mean_img_0, mean_img_1], titles=[labels_map[0],
labels_map[1]], ncols=2, figsize=(6,3))

# Hint for (2): pick 5 indices near the center of PCA space
center_idx = np.argsort((X_pca[:,0]**2 + X_pca[:,1]**2))[:5]
imgs_to_show = []
tits = []
for ii in center_idx:
    img_vec = X[ii].reshape(28,28)
    imgs_to_show.append(img_vec)
    tits.append(labels_map[int(y[ii])])
show_image_grid(imgs_to_show, titles=tits, ncols=5, figsize=(12,3))
```



End of Chunk 2 — what we covered

- Loading Fashion-MNIST and inspecting image tensors.
- Building a light pandas DataFrame with per-image metadata and practicing `groupby`, `describe`, and handling missing values.
- Visualizing class distributions, mean/std features, and using PCA to visualize high-dimensional image data in 2D.

- Small feature-engineering exercises which bridge to model-building in the next chunk.

Next (Chunk 3) we'll build a quick scikit-learn pipeline (train/test split, scaling, a logistic regression / random forest baseline), evaluate models using classification metrics, and introduce cross-validation.

Chunk 3 — Quick scikit-learn pipeline: baseline models, cross-validation & metrics

Goals for this chunk:

- Build fast baselines using scikit-learn on flattened image vectors.
- Demonstrate `Pipeline` (scaling → dimensionality reduction → classifier).
- Compare Logistic Regression vs Random Forest as simple baselines.
- Evaluate with accuracy, per-class metrics, confusion matrix, ROC and PR curves (one-vs-rest).
- Introduce cross-validation and hyperparameter search (GridSearchCV).

Note: We operate on a reduced subset (2k samples) to keep compute interactive. In practice you'd use larger training sets and consider GPU-based training for deep models.

```
# Additional sklearn imports for this chunk
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, LabelBinarizer
from sklearn.model_selection import train_test_split, cross_val_score,
GridSearchCV, StratifiedKFold
from sklearn.metrics import (
    ConfusionMatrixDisplay, RocCurveDisplay, PrecisionRecallDisplay,
    accuracy_score, classification_report
)

# # Helper: plot confusion matrix
# def plot_confusion_matrix(y_true, y_pred, labels=None,
normalize='true', figsize=(6,6), cmap=None):
#     cm = confusion_matrix(y_true, y_pred, normalize=normalize)
#     disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=labels)
#     fig, ax = plt.subplots(figsize=figsize)
#     disp.plot(ax=ax, cmap=cmap if cmap is not None else 'viridis',
colorbar=True)
#     plt.title("Confusion Matrix
(normalized='{}')".format(normalize))
#     plt.show()

def plot_confusion_matrix(
    y_true,
    y_pred,
```

```
    labels=None,
    normalize='true',       # same options as
sklearn.metrics.confusion_matrix: None, 'true', 'pred', 'all'
    figsize=(8, 6),
    cmap=None,              # colormap name or object; default 'Blues'
    colorbar=True,
    rotation=45,            # x-tick rotation (deg)
    ha='right',             # horizontal alignment for x-tick labels
    fontsize=10,            # tick label font size
    title=None,             # custom title (if None, a default title
is used)
    annotate=True           # whether to show values on the matrix
display
):
    """
    Nicely formatted confusion matrix display that avoids overlapping
labels.
    Returns (fig, ax) so caller can further modify the plot.
    """
    if cmap is None:
        cmap = 'Blues'

    # If labels not provided, infer from the union of true/pred sorted
    if labels is None:
        unique_labels = np.unique(np.concatenate([np.asarray(y_true),
np.asarray(y_pred)]))
        labels = [str(l) for l in unique_labels]

    # Compute confusion matrix (allows normalization)
    cm = confusion_matrix(y_true, y_pred, normalize=normalize)

    # Create display
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=labels)
    fig, ax = plt.subplots(figsize=figsize)

    # Use built-in plot; pass xticks_rotation to help with label
overlap
    disp.plot(
        ax=ax,
        cmap=cmap,
        colorbar=colorbar,
        include_values=annotate,
        xticks_rotation=rotation
    )

    # Tidy tick label alignment & font size (reinforce alignment in
case backend ignored xticks_rotation)
    ax.set_xticklabels(ax.get_xticklabels(), rotation=rotation, ha=ha,
fontsize=fontsize)
```

```
    ax.set_yticklabels(ax.get_yticklabels(), fontsize=fontsize)

    # Title
    if title is None:
        ax.set_title(f"Confusion Matrix (normalized='{normalize}')")
    else:
        ax.set_title(title)

    plt.tight_layout()
    plt.show()
    return fig, ax

# Helper: plot ROC curves for multiclass (one-vs-rest)
def plot_multiclass_roc(y_true, y_score, n_classes, labels_map=None):
    """
    y_true: (n,) integer labels
    y_score: (n, n_classes) predicted scores / probabilities
    """
    from sklearn.preprocessing import label_binarize
    y_bin = label_binarize(y_true, classes=list(range(n_classes)))
    plt.figure(figsize=(8,6))
    for i in range(n_classes):
        fpr, tpr, _ = roc_curve(y_bin[:, i], y_score[:, i])
        plt.plot(fpr, tpr, lw=2, label=f"Class {i}" if labels_map is
None else labels_map[i])
    plt.plot([0,1],[0,1],'k--', lw=1)
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("Multi-class ROC (one-vs-rest)")
    plt.legend(loc='lower right', fontsize=8)
    plt.show()
```

Train/test split and a baseline with Logistic Regression

We'll split the flattened data `X` and labels `y` into train and test sets (80/20), then create a pipeline: `StandardScaler` → `PCA (optional)` → `LogisticRegression`.

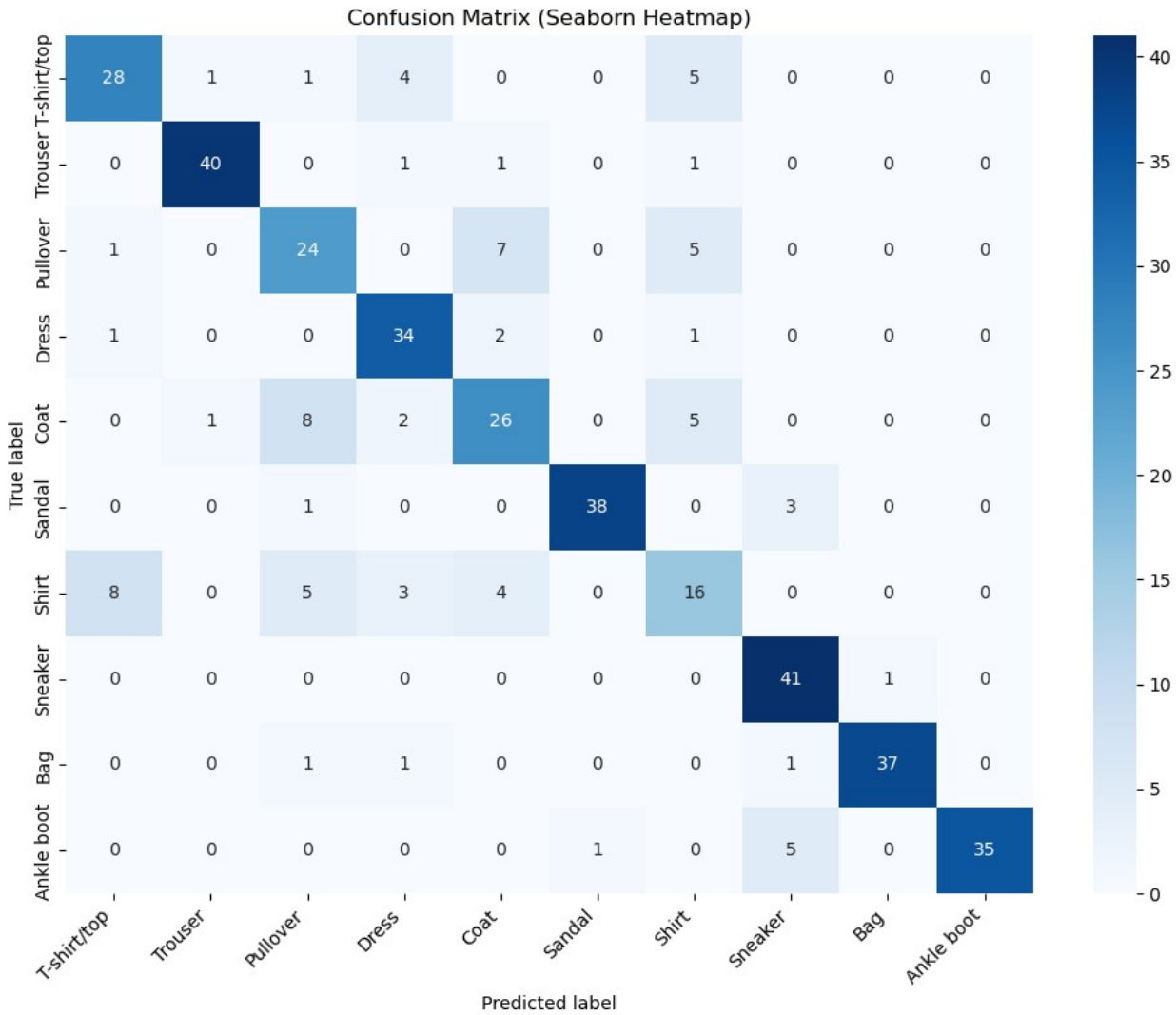
Using PCA before the classifier reduces dimensionality and often improves both speed and conditioning for linear models.

```
# Confirm X and y exist (created in Chunk 2)
print("X shape:", X.shape, " y shape:", y.shape, "unique labels:",
np.unique(y))

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=42)
print("Train:", X_train.shape, "Test:", X_test.shape)
```

```python
# We'll use a small PCA for speed (n_components=50). You can set None
to skip PCA.
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report,
ConfusionMatrixDisplay
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Example pipeline
logreg_pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('logreg', LogisticRegression(max_iter=1000))
])

logreg_pipe.fit(X_train, y_train)
y_pred = logreg_pipe.predict(X_test)

# Corrected print statement
print("Baseline Logistic Regression accuracy:",
round(accuracy_score(y_test, y_pred), 4))
print("\nClassification report:\n", classification_report(y_test,
y_pred, target_names=[labels_map[i] for i in range(10)]))

# ConfusionMatrixDisplay.from_predictions(y_test, y_pred,
display_labels=[labels_map[i] for i in range(10)], cmap='Blues')
# plt.show()

cm = confusion_matrix(y_test, y_pred)
labels = [labels_map[i] for i in range(10)]

plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.title("Confusion Matrix (Seaborn Heatmap)")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
X shape: (2000, 784)  y shape: (2000,) unique labels: [0 1 2 3 4 5 6 7
8 9]
Train: (1600, 784) Test: (400, 784)
Baseline Logistic Regression accuracy: 0.7975

Classification report:
               precision    recall  f1-score   support
```

T-shirt/top	0.74	0.72	0.73	39
Trouser	0.95	0.93	0.94	43
Pullover	0.60	0.65	0.62	37
Dress	0.76	0.89	0.82	38
Coat	0.65	0.62	0.63	42
Sandal	0.97	0.90	0.94	42
Shirt	0.48	0.44	0.46	36
Sneaker	0.82	0.98	0.89	42
Bag	0.97	0.93	0.95	40
Ankle boot	1.00	0.85	0.92	41
accuracy			0.80	400
macro avg	0.79	0.79	0.79	400
weighted avg	0.80	0.80	0.80	400

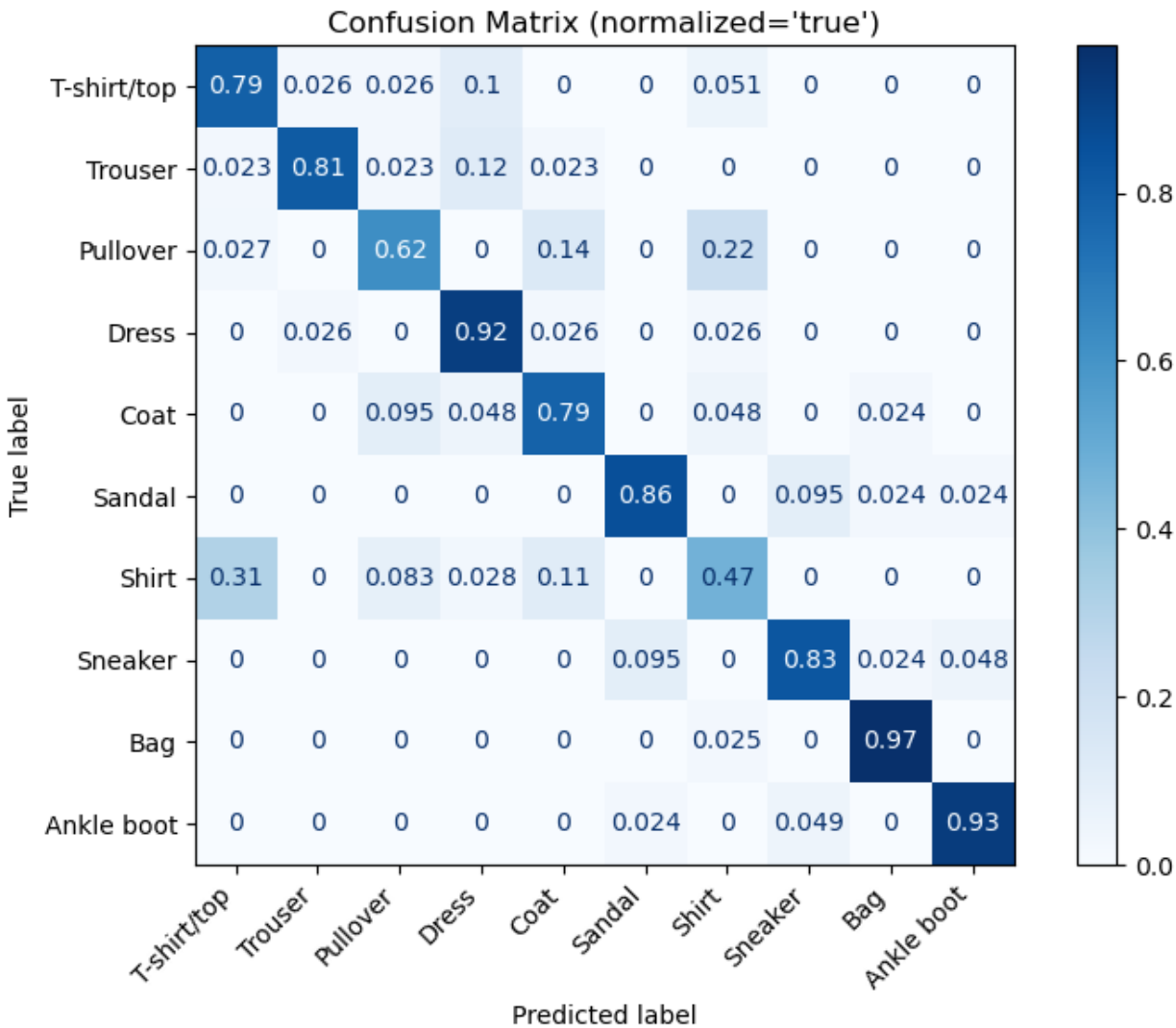


Random Forest baseline

Random Forests often perform well with minimal tuning on tabular/flattened data. They can capture non-linearities absent in a linear model. Here we'll run a moderate-size forest (100 trees) for comparison.

```
rf_pipe = Pipeline([
    ("scaler", StandardScaler()),  # RF doesn't strictly need scaling,
but kept for consistent preprocessing
    ("pca", PCA(n_components=50, random_state=42)),
    ("clf", RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1))
])

rf_pipe.fit(X_train, y_train)
y_pred_rf = rf_pipe.predict(X_test)
print("Random Forest accuracy:", round(accuracy_score(y_test,
y_pred_rf), 4))
print("\nClassification report:\n", classification_report(y_test,
y_pred_rf, target_names=[labels_map[i] for i in range(10)]))
plot_confusion_matrix(y_test, y_pred_rf, labels=[labels_map[i] for i
in range(10)])
```

```
Random Forest accuracy: 0.805

Classification report:
               precision    recall  f1-score   support

 T-shirt/top       0.70      0.79      0.75        39
     Trouser       0.95      0.81      0.88        43
    Pullover       0.72      0.62      0.67        37
       Dress       0.74      0.92      0.82        38
        Coat       0.75      0.79      0.77        42
      Sandal       0.88      0.86      0.87        42
       Shirt       0.55      0.47      0.51        36
     Sneaker       0.85      0.83      0.84        42
         Bag       0.93      0.97      0.95        40
  Ankle boot       0.93      0.93      0.93        41

    accuracy                           0.81       400
   macro avg       0.80      0.80      0.80       400
weighted avg       0.81      0.81      0.80       400
```




```
(<Figure size 800x600 with 2 Axes>,
 <Axes: title={'center': "Confusion Matrix (normalized='true')"},
xlabel='Predicted label', ylabel='True label'>)
```

Cross-validation & GridSearchCV (quick)

We'll run a small GridSearchCV on the logistic regression pipeline to find a good regularization strength `C`. Use `StratifiedKFold` to preserve class balance. Keep the grid tiny so it's interactive.

```
from sklearn.model_selection import StratifiedKFold, GridSearchCV

# Correct param grid: use the pipeline step name 'logreg'
param_grid = {
    "logreg__C": [0.01, 0.1, 1.0]
}
```

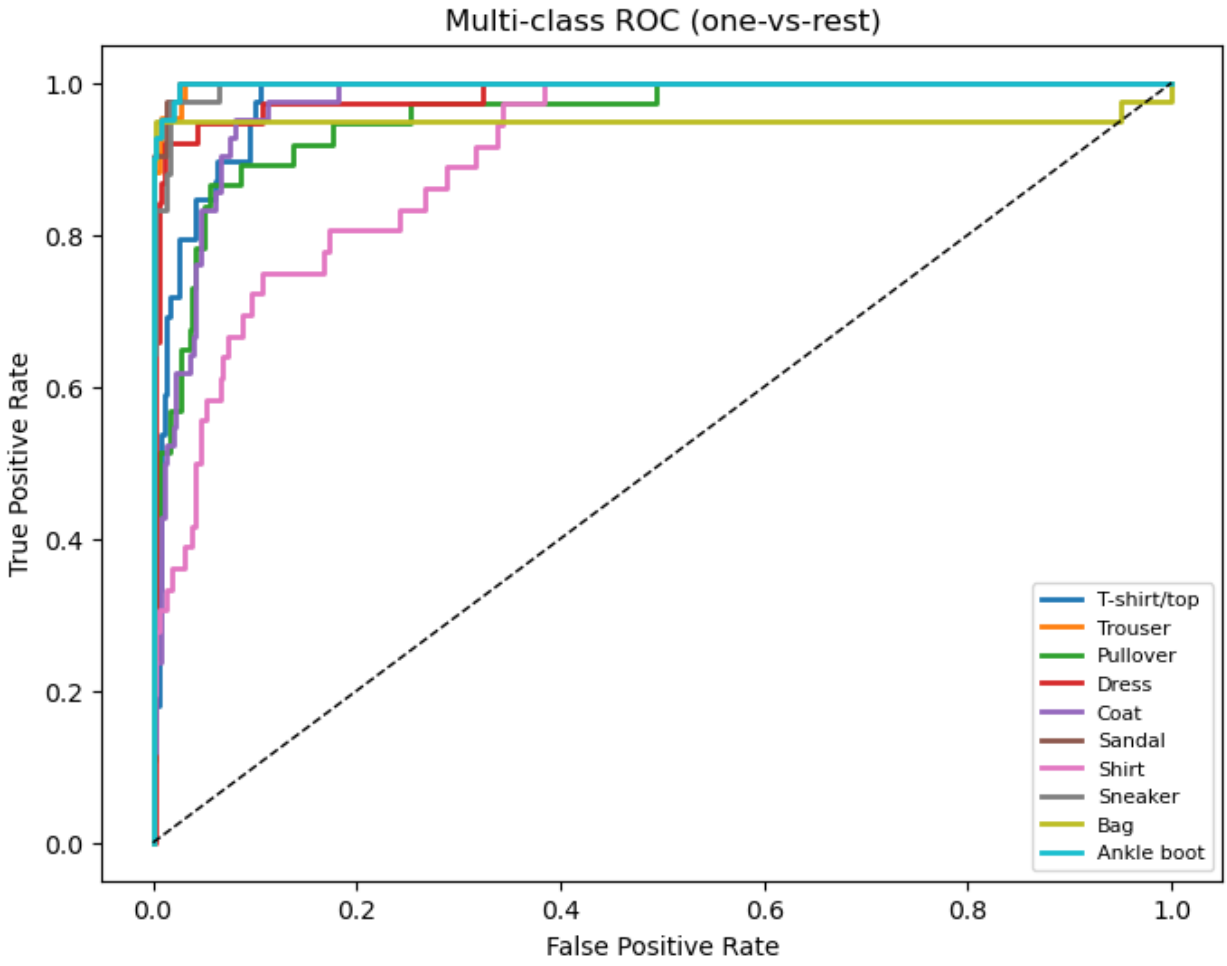
```
cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=42)
grid = GridSearchCV(logreg_pipe, param_grid=param_grid, cv=cv,
n_jobs=-1, verbose=1, scoring='accuracy', refit=True)

grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)
print("Best CV score:", round(grid.best_score_, 4))

best_model = grid.best_estimator_
y_pred_best = best_model.predict(X_test)
print("Test accuracy (best model):", round(accuracy_score(y_test,
y_pred_best), 4))
```

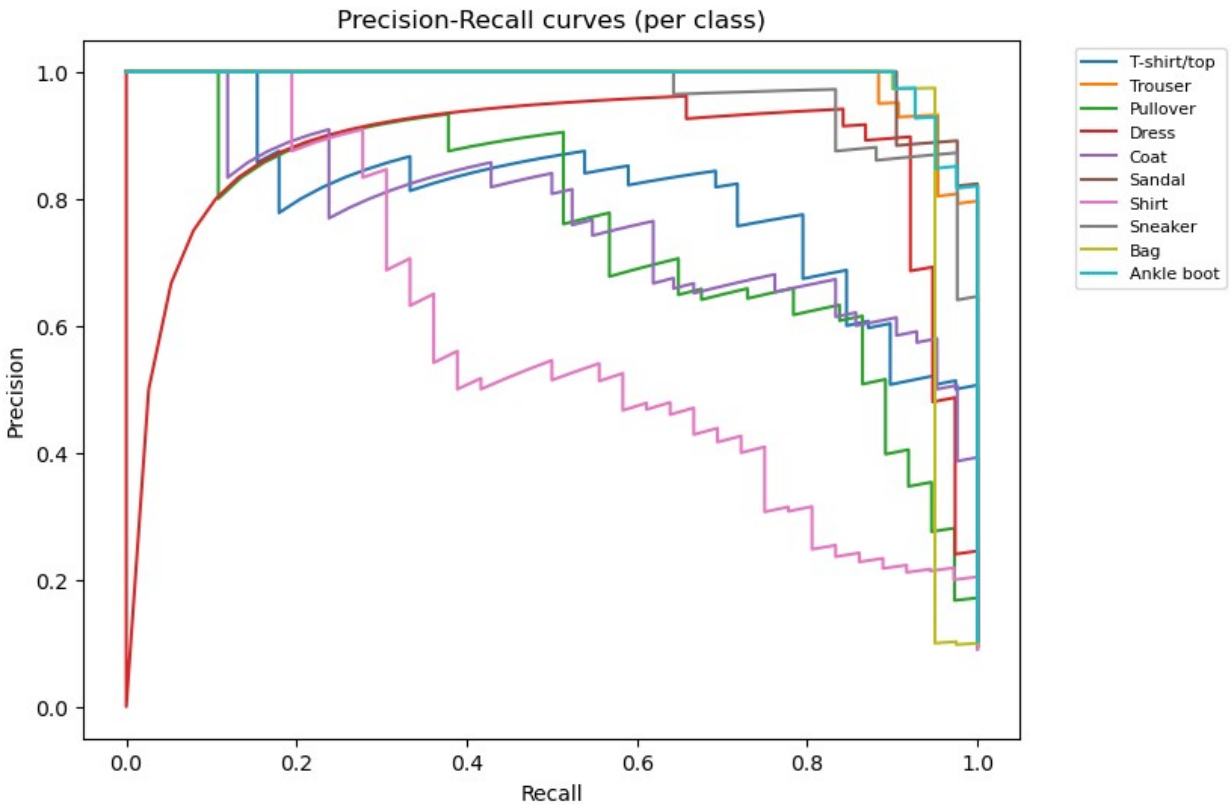
```
Fitting 4 folds for each of 3 candidates, totalling 12 fits
Best params: {'logreg__C': 0.1}
Best CV score: 0.8138
Test accuracy (best model): 0.8075
```

ROC and Precision-Recall (PR) curves for multiclass

For multiclass problems we use a one-vs-rest approach: compute per-class ROC using binarized labels. These curves provide insight beyond plain accuracy — for example sensitivity at specific false-positive rates — which is important in medical tasks.

```
# Need predicted probabilities
y_score = best_model.predict_proba(X_test)  # shape (n_samples,
n_classes)
n_classes = y_score.shape[1]

# Plot ROC curves
plot_multiclass_roc(y_test, y_score, n_classes=n_classes,
labels_map=labels_map)

# Plot PR curves for a subset of classes (too many curves can clutter)
plt.figure(figsize=(8,6))
from sklearn.preprocessing import label_binarize
y_bin = label_binarize(y_test, classes=list(range(n_classes)))
for i in range(n_classes):
    precision, recall, _ = precision_recall_curve(y_bin[:, i],
y_score[:, i])
    plt.plot(recall, precision, label=labels_map[i])
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall curves (per class)")
plt.legend(bbox_to_anchor=(1.05,1), loc='upper left', fontsize=8)
plt.show()
```


Multi-class ROC (one-vs-rest)
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
T-shirt/top
Trouser
Pullover
Dress
Coat
Sandal
Shirt
Sneaker
Bag
Ankle boot



Model selection tips & pitfalls

- **Overfitting**: A model that performs much better on train than test is likely overfitting. Use cross-validation to estimate generalization.
- **Data leakage**: Never fit scalers/PCA on the full dataset before splitting — use Pipeline so transforms are fit only on training data.
- **Imbalanced classes**: Accuracy can be misleading. Use per-class metrics, AUC, precision/recall or class-weighted losses.
- **Speed vs performance**: Simple models (logistic/regression) are fast and interpretable; ensembles or deep models may improve accuracy but cost more compute.
- **Reproducibility**: set `random_state` everywhere to get stable results during teaching. In production, don't forget to monitor drift.

Exercise 4 (hands-on): Modeling tasks

1. Compare model performance (accuracy & AUC) between:
   - Logistic Regression with PCA (n_components = 20, 50, 100)
   - Random Forest (n_estimators = 50, 100) Report results in a concise table.
2. Investigate per-class errors:
   - For the best model, list the 5 most confused class pairs (based on confusion matrix).
   - For one confused pair (e.g., "Shirt" vs "T-shirt/top"), visualize 10 examples from each class and comment on possible reasons for confusion.
3. (Optional, advanced) Train a small linear SVM (`sklearn.svm.LinearSVC`) on the PCA features and compare.

End of Chunk 3

What we accomplished:

- Built reproducible scikit-learn pipelines for flattened image data.
- Compared linear (Logistic Regression) and non-linear (Random Forest) baselines.
- Used cross-validation and GridSearchCV for simple hyperparameter tuning.
- Visualized model behavior via confusion matrices, ROC, and PR curves.

Next (Chunk 4) we'll switch gears to **PyTorch**: tensors, autograd, `Dataset`/`DataLoader`, and a simple gradient example — preparing us to implement a small CNN training loop in Chunk 5.

Chunk 4 — PyTorch essentials: tensors, autograd, Dataset & DataLoader

Goals:

- Understand the difference between NumPy arrays and PyTorch tensors.
- Learn how `requires_grad` and autograd compute gradients automatically.
- Practice a tiny manual optimization step (no `nn.Module`) to see how autograd works under the hood.
- Learn how to use `torch.utils.data.Dataset` and `DataLoader` (including transforms and batching).
- Confirm tensor shapes and device handling so your training loop (Chunk 5) runs smoothly.

Notes for students:

- PyTorch uses **channels-first** image format `(C, H, W)`. Most torchvision transforms produce tensors in that layout.
- When debugging shape mistakes, print `.shape` early and often.

```
# Create tensors and arrays
arr = np.arange(12).reshape(3,4).astype(np.float32)
t = torch.tensor(arr)                # from numpy -> copy by default
t2 = torch.from_numpy(arr)           # shares memory with numpy array
if dtypes match
t_rand = torch.rand(3,4)             # random tensor

print("numpy arr shape:", arr.shape, " dtype:", arr.dtype)
print("torch tensor t shape:", t.shape, " dtype:", t.dtype, "
device:", t.device)
print("torch random t_rand:\n", t_rand)

# Convert back to numpy
np_from_t = t.numpy()
print("np_from_t shape:", np_from_t.shape)

# Device example: move tensor to GPU if available
```

```
if torch.cuda.is_available():
    t_gpu = t.to('cuda')
    print("Moved tensor to GPU:", t_gpu.device)
else:
    print("CUDA not available; using CPU.")
```

```
numpy arr shape: (3, 4)  dtype: float32
torch tensor t shape: torch.Size([3, 4])  dtype: torch.float32
device: cpu
torch random t_rand:
 tensor([[0.8823, 0.9150, 0.3829, 0.9593],
        [0.3904, 0.6009, 0.2566, 0.7936],
        [0.9408, 0.1332, 0.9346, 0.5936]])
np_from_t shape: (3, 4)
CUDA not available; using CPU.
```

Autograd intuition

- When `tensor.requires_grad == True`, PyTorch builds a computation graph as operations happen.
- Calling `.backward()` on a scalar output will compute gradients for all tensors with `requires_grad=True` in that graph.
- Gradients accumulate in `.grad` attributes (so typically call `optimizer.zero_grad()` or `param.grad.zero_()` before backprop).

We'll demonstrate with a small scalar function and compute derivatives.

```
# Scalar example: y = 3*x^2 + 2*x
x = torch.tensor([2.0], requires_grad=True)
y = 3 * x**2 + 2 * x
print("y:", y.item())

# Backprop: dy/dx = 6*x + 2
y.backward()
print("dy/dx at x=2 ->", x.grad.item())  # expected 6*2 + 2 = 14

# Vector example: compute gradient of scalar output that depends on a
vector parameter
w = torch.randn(5, requires_grad=True)
inp = torch.randn(5)
loss = torch.sum((w * inp)**2)  # scalar
loss.backward()
print("w.grad shape:", w.grad.shape)
```

```
y: 16.0
dy/dx at x=2 -> 14.0
w.grad shape: torch.Size([5])
```

Manual optimization step (educational)

Before `torch.optim` existed, people manually updated parameters using gradients:

We'll do a toy linear-regression-style update to illustrate the algebraic flow. (We will use `torch.optim` for real training in Chunk 5.)

```
# Toy linear regression: y = w*x + b, optimize w and b to fit a small synthetic dataset
torch.manual_seed(0)
X = torch.randn(50, 1)
true_w, true_b = 2.5, -1.0
y_true = true_w * X + true_b + 0.1 * torch.randn_like(X)

# initialize parameters
w = torch.randn(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
lr = 0.1

for epoch in range(20):
    y_pred = X * w + b            # forward
    loss = torch.mean((y_pred - y_true)**2)  # MSE
    loss.backward()               # compute gradients
    with torch.no_grad():         # disable grad while updating params
        w -= lr * w.grad
        b -= lr * b.grad
    # zero gradients for next iteration
    w.grad.zero_()
    b.grad.zero_()
    if epoch % 5 == 0:
        print(f"Epoch {epoch} loss: {loss.item():.4f}")

print("Learned w,b:", w.item(), b.item(), " True:", true_w, true_b)
```

```
Epoch 0 loss: 19.7459
Epoch 5 loss: 1.5405
Epoch 10 loss: 0.1340
Epoch 15 loss: 0.0201
Learned w,b: 2.4685561656951904 -0.9673100709915161  True: 2.5 -1.0
```

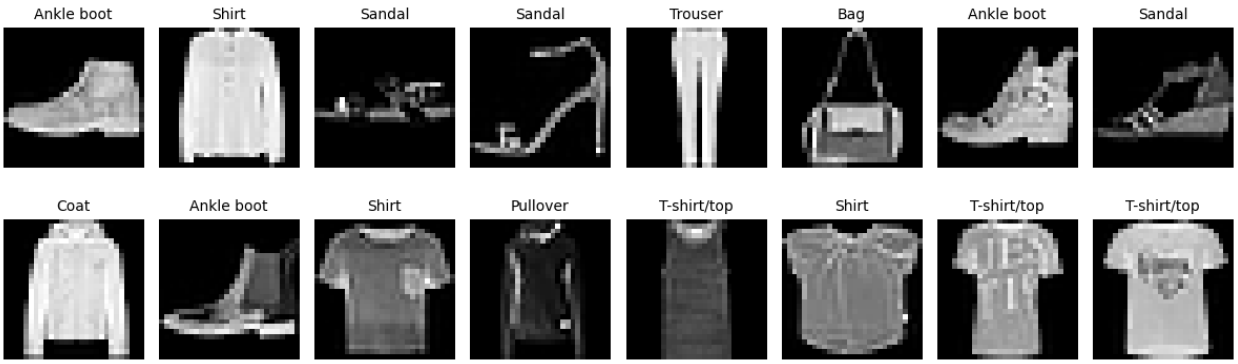
Dataset & DataLoader overview

- `Dataset` is an object that implements `__len__` and `__getitem__`. It returns one sample (input, label).
- `DataLoader` wraps a `Dataset` and provides batching, shuffling, and multi-process loading.
- `transform` functions often live in the `Dataset` (e.g., `torchvision.transforms`) to do augmentation and normalization.
- For images, a typical pipeline:

- Load PIL image → apply transforms (resize, to tensor, normalize) → tensor shape (C, H, W)

```
# Reuse Fashion-MNIST train_ds from earlier; create a DataLoader for
mini-batches
batch_size = 64
# Define transforms: ToTensor (scales 0-255 to 0-1) and normalize
(mean,std) for single-channel dataset
fm_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.2860,), std=(0.3530,))  # approximate
stats for FashionMNIST
])

train_ds_transformed = datasets.FashionMNIST(root=data_dir,
train=True, download=False, transform=fm_transform)
train_loader = DataLoader(train_ds_transformed, batch_size=batch_size,
shuffle=True, num_workers=0)

# Inspect one batch
images, labels = next(iter(train_loader))
print("Batch image tensor shape:", images.shape)  # expected (B, C, H,
W)
print("Batch labels shape:", labels.shape)
print("dtype:", images.dtype, "min/max:", images.min().item(),
images.max().item())

# Visualize first 16 images of the batch (denormalize for display)
def denormalize(img_tensor, mean=0.2860, std=0.3530):
    return img_tensor * std + mean

grid_imgs = [denormalize(images[i]).squeeze(0).numpy() for i in
range(min(16, images.shape[0]))]
grid_titles = [labels_map[int(l)] for l in labels[:16]]
show_image_grid(grid_imgs, titles=grid_titles, ncols=8,
figsize=(12,4))
```

```
Batch image tensor shape: torch.Size([64, 1, 28, 28])
Batch labels shape: torch.Size([64])
dtype: torch.float32 min/max: -0.8101983666419983 2.022662878036499
```



Custom Dataset example

Sometimes data doesn't come in torchvision format. Here's a template for a `Dataset` that:

- Accepts a list/array of image arrays (H,W or H,W,C) or paths.
- Applies transforms (if provided).
- Returns (image_tensor, label) ready for DataLoader.

This is a reusable pattern for CSV/NumPy-based datasets and for cases where you have separate metadata.

```
from PIL import Image

class NumpyImageDataset(torch.utils.data.Dataset):
    """
    Simple Dataset for in-memory numpy images and labels.
    images: array-like of shape (N, H, W) or (N, H, W, C)
    labels: list/array of length N
    transform: torchvision transform to apply (expects PIL or tensor
depending on transform)
    """
    def __init__(self, images, labels, transform=None, to_pil:
bool=False):
        self.images = images
        self.labels = labels
        self.transform = transform
        self.to_pil = to_pil

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        img = self.images[idx]
        # optionally convert to PIL for transforms that expect PIL
images
        if self.to_pil:
            img = Image.fromarray((img * 255).astype('uint8')) if
img.dtype != np.uint8 else Image.fromarray(img)
```

```
        if self.transform is not None:
            img = self.transform(img)
        else:
            # ensure returned tensor is float in [0,1], shape (C,H,W)
            if isinstance(img, np.ndarray):
                if img.ndim == 2:
                    img = torch.from_numpy(img).unsqueeze(0).float() /
255.0
                elif img.ndim == 3:
                    img = torch.from_numpy(img).permute(2,0,1).float()
/ 255.0
        label = int(self.labels[idx])
        return img, label

# quick sanity check: create dataset from a few fashion-mnist images
(convert to numpy)
sample_imgs = [train_ds[i][0].squeeze(0).numpy() for i in range(100)]
sample_labels = [int(train_ds[i][1]) for i in range(100)]
np_dataset = NumpyImageDataset(sample_imgs, sample_labels,
transform=transforms.Compose([transforms.ToTensor()]))
np_loader = DataLoader(np_dataset, batch_size=16, shuffle=False)
imgs_np, labs_np = next(iter(np_loader))
print("np_dataset batch shapes:", imgs_np.shape, labs_np.shape)
```

```
np_dataset batch shapes: torch.Size([16, 1, 28, 28]) torch.Size([16])
```

Collate function (advanced note)

- By default DataLoader collates samples into a batch by stacking tensors along the 0th dim.
- For variable-sized inputs (e.g., variable image sizes, or text sequences), provide a custom `collate_fn` to pad or handle lists of varying sizes.
- Example use-cases: variable-length audio clips, sequences, or detections per image.

```
# Minimal nn.Module demonstrating forward and backward on a batch
import torch.nn as nn
torch.manual_seed(0)

class TinyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )
    def forward(self, x):
        return self.net(x)
```

```python
model = TinyNet().to(DEVICE)
# take one batch from train_loader and move to DEVICE (denormalize not
necessary for training)
imgs_batch, labels_batch = next(iter(train_loader))
imgs_batch = imgs_batch.to(DEVICE)
labels_batch = labels_batch.to(DEVICE)

# forward
logits = model(imgs_batch)
print("Logits shape:", logits.shape)  # (B, num_classes)

# compute loss and backward
criterion = nn.CrossEntropyLoss()
loss = criterion(logits, labels_batch)
print("Loss item:", loss.item())
loss.backward()  # computes gradients for model parameters
# check one gradient
for name, param in model.named_parameters():
    if param.grad is None:
        continue
    print(name, " grad mean:", param.grad.mean().item())
    break

# Zero grads (we won't do optimizer.step() here; full training in
Chunk 5)
model.zero_grad()
```

```
Logits shape: torch.Size([64, 10])
Loss item: 2.3070008754730225
net.1.weight  grad mean: 0.00026585644809529185
```

Common debugging checklist for PyTorch

- Are your tensors on the same device? (`.to(DEVICE)`).
- Are shapes what you expect? Print `.shape` on inputs, logits, labels.
- Is `nn.CrossEntropyLoss` used with raw logits (no softmax)? It expects class indices as targets.
- Are gradients `None`? If yes, maybe no `requires_grad=True` parameter, or you used `torch.no_grad()` accidentally.
- Did you forget to call `optimizer.zero_grad()` before `loss.backward()` in a real training loop (you'll zero before the next step or after the update)?
- Floating-point precision: prefer `float32` for training; conversions from `uint8` must be handled before `nn.Linear`.

Exercise 5 (hands-on): PyTorch practice

1. Convert a small subset of Fashion-MNIST (200 samples) into a `NumpyImageDataset`, applying random horizontal flip augmentation and normalization. Create a DataLoader and visualize a batch.

2. Using the `TinyNet` above:
   - Move it to `DEVICE`.
   - Run a single optimizer step (use `torch.optim.Adam`) on one batch: `forward -> compute loss -> backward -> optimizer.step()` and print loss before & after the update.
   - Inspect and report the L2 norm of gradients for the first linear layer before and after `optimizer.step()` (remember gradients are cleared after step only if you call `zero_grad()`; typically you zero before the next backward).
3. (Optional) Modify `TinyNet` to include a dropout layer and observe whether the output logits change from one forward pass to the next with `model.train()` vs `model.eval()`.

End of Chunk 4

What we covered:

- Torch vs NumPy conversions and device handling.
- Autograd basics and a manual parameter update example.
- `Dataset` / `DataLoader` patterns including a custom `NumpyImageDataset` template.
- Single forward/backward with a small `nn.Module` to validate shapes and gradients.
- Debugging checklist and practical exercises.

Next (Chunk 5) we'll implement a complete small CNN, the training loop (metrics & plotting), model checkpointing (save/load), evaluation (confusion matrix), and finally extracting embeddings from a pretrained ResNet as a bridge to Notebook 2.

Chunk 5 — Small CNN: training loop, checkpointing, evaluation & embeddings

Goals:

- Implement a complete training loop for a small CNN (train / val / test).
- Track loss & accuracy and plot learning curves.
- Save and load model checkpoints.
- Evaluate by computing confusion matrix and per-class metrics.
- Extract embeddings from a pretrained ResNet (feature extractor) to demonstrate retrieval with k-NN.
- Measure a simple inference latency estimate (single-batch forward).

```
import time
import copy
import torch.nn as nn
import torch.optim as optim
from tqdm.notebook import tqdm

# Reuse DEVICE from earlier
print("Device:", DEVICE)
```

```
# Training hyperparameters (kept small for classroom demo)
num_epochs = 5
batch_size = 128
learning_rate = 1e-3
checkpoint_dir = "./checkpoints"
os.makedirs(checkpoint_dir, exist_ok=True)

# Prepare DataLoaders: train/val/test splits built from torchvision
dataset
# We'll use the train split from torchvision and carve out a small
validation set
full_train_ds = datasets.FashionMNIST(root=data_dir, train=True,
download=False, transform=fm_transform)
test_ds = datasets.FashionMNIST(root=data_dir, train=False,
download=False, transform=fm_transform)

# create train/val split
n_full = len(full_train_ds)
val_fraction = 0.1
n_val = int(n_full * val_fraction)
n_train = n_full - n_val
train_ds_small, val_ds = torch.utils.data.random_split(full_train_ds,
[n_train, n_val], generator=torch.Generator().manual_seed(42))

train_loader = DataLoader(train_ds_small, batch_size=batch_size,
shuffle=True, num_workers=0)
val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False,
num_workers=0)
test_loader = DataLoader(test_ds, batch_size=batch_size,
shuffle=False, num_workers=0)

print("Train samples:", len(train_ds_small), "Val samples:",
len(val_ds), "Test samples:", len(test_ds))
```

```
Device: cpu
Train samples: 54000 Val samples: 6000 Test samples: 10000
```

We'll define a compact CNN suitable for 28×28 grayscale images. It's intentionally small so students can run training quickly on CPU or a basic GPU.

```
class SmallCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),   #
(B,32,28,28)
            nn.ReLU(),
            nn.MaxPool2d(2),                              #
(B,32,14,14)
```

```
            nn.Conv2d(32, 64, kernel_size=3, padding=1),  #
(B,64,14,14)
            nn.ReLU(),
            nn.MaxPool2d(2),                              # (B,64,7,7)
            nn.Conv2d(64, 128, kernel_size=3, padding=1), #
(B,128,7,7)
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,1))                   #
(B,128,1,1)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, num_classes)
        )
    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

# instantiate model
model = SmallCNN(num_classes=10).to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
```

Training loop outline:

1. For each epoch:
   - Train phase: model.train(), iterate train_loader, forward → loss → backward → optimizer.step()
   - Validation phase: model.eval(), no_grad, compute metrics on val_loader
2. Track training and validation loss & accuracy per epoch for plotting.
3. Keep a best-model copy (by validation accuracy) and save checkpoints to disk.

```
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    running_corrects = 0
    n_samples = 0
    for inputs, labels in loader:
        inputs = inputs.to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

```
        preds = outputs.argmax(dim=1)
        running_loss += loss.item() * inputs.size(0)
        running_corrects += (preds == labels).sum().item()
        n_samples += inputs.size(0)
    epoch_loss = running_loss / n_samples
    epoch_acc = running_corrects / n_samples
    return epoch_loss, epoch_acc

def evaluate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    running_corrects = 0
    n_samples = 0
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for inputs, labels in loader:
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            preds = outputs.argmax(dim=1)
            running_loss += loss.item() * inputs.size(0)
            running_corrects += (preds == labels).sum().item()
            n_samples += inputs.size(0)
            all_preds.append(preds.cpu().numpy())
            all_labels.append(labels.cpu().numpy())
    epoch_loss = running_loss / n_samples
    epoch_acc = running_corrects / n_samples
    all_preds = np.concatenate(all_preds)
    all_labels = np.concatenate(all_labels)
    return epoch_loss, epoch_acc, all_preds, all_labels

# training loop with checkpointing
history = {"train_loss":[], "train_acc":[], "val_loss":[], "val_acc":
[]}
best_val_acc = 0.0
best_model_wts = copy.deepcopy(model.state_dict())
start_time = time.time()

for epoch in range(1, num_epochs+1):
    train_loss, train_acc = train_one_epoch(model, train_loader,
criterion, optimizer, DEVICE)
    val_loss, val_acc, _, _ = evaluate(model, val_loader, criterion,
DEVICE)
    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["val_loss"].append(val_loss)
    history["val_acc"].append(val_acc)
    print(f"Epoch {epoch}/{num_epochs}  train_loss: {train_loss:.4f}
```

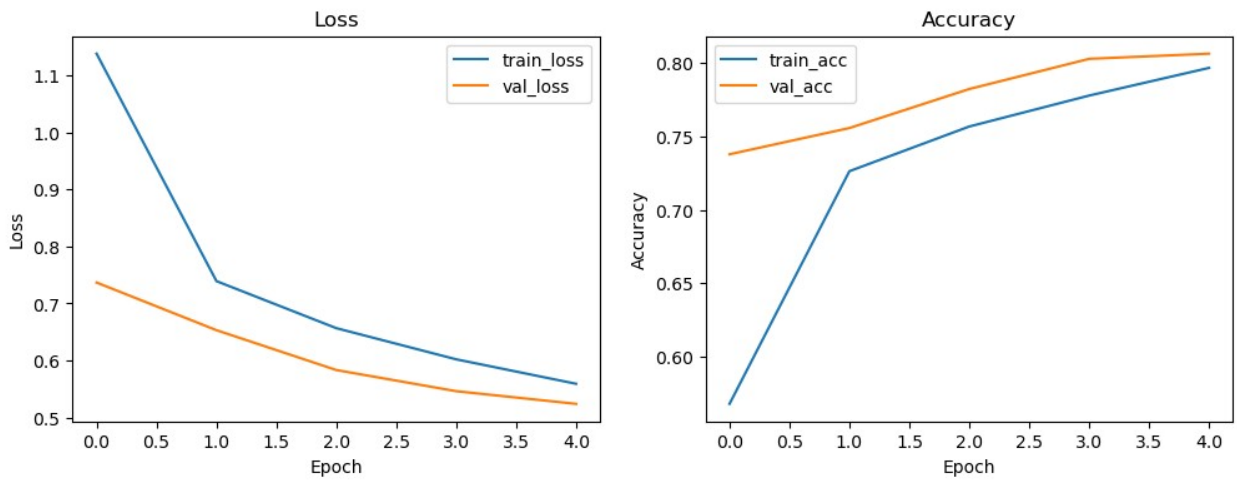
```
train_acc: {train_acc:.4f}  val_loss: {val_loss:.4f} val_acc:
{val_acc:.4f}")
    # checkpoint if improved
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_model_wts = copy.deepcopy(model.state_dict())
        checkpoint_path = os.path.join(checkpoint_dir,
f"best_model_epoch{epoch}_valacc{val_acc:.4f}.pth")
        torch.save({
            "epoch": epoch,
            "model_state_dict": model.state_dict(),
            "optimizer_state_dict": optimizer.state_dict(),
            "val_acc": val_acc
        }, checkpoint_path)
        print("Saved checkpoint:", checkpoint_path)

elapsed = time.time() - start_time
print(f"Training complete in {elapsed/60:.2f} minutes. Best val acc:
{best_val_acc:.4f}")

# load best weights back into model for evaluation
model.load_state_dict(best_model_wts)
```

```
Epoch 1/5  train_loss: 0.3207 train_acc: 0.8880  val_loss: 0.3037
val_acc: 0.8898
Saved checkpoint: ./checkpoints\best_model_epoch1_valacc0.8898.pth
Epoch 2/5  train_loss: 0.3102 train_acc: 0.8919  val_loss: 0.3163
val_acc: 0.8847
Epoch 3/5  train_loss: 0.3001 train_acc: 0.8939  val_loss: 0.2739
val_acc: 0.9002
Saved checkpoint: ./checkpoints\best_model_epoch3_valacc0.9002.pth
Epoch 4/5  train_loss: 0.2924 train_acc: 0.8970  val_loss: 0.2702
val_acc: 0.9007
Saved checkpoint: ./checkpoints\best_model_epoch4_valacc0.9007.pth
Epoch 5/5  train_loss: 0.2814 train_acc: 0.9018  val_loss: 0.2795
val_acc: 0.8973
Training complete in 2.87 minutes. Best val acc: 0.9007

<All keys matched successfully>
```

Plot training and validation loss/accuracy over epochs to inspect under/overfitting behavior.

```
plt.figure(figsize=(12,4))
plt.subplot(1,2,1)
plt.plot(history["train_loss"], label="train_loss")
plt.plot(history["val_loss"], label="val_loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.title("Loss")
```

```
plt.subplot(1,2,2)
plt.plot(history["train_acc"], label="train_acc")
plt.plot(history["val_acc"], label="val_acc")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Accuracy")
plt.show()
```

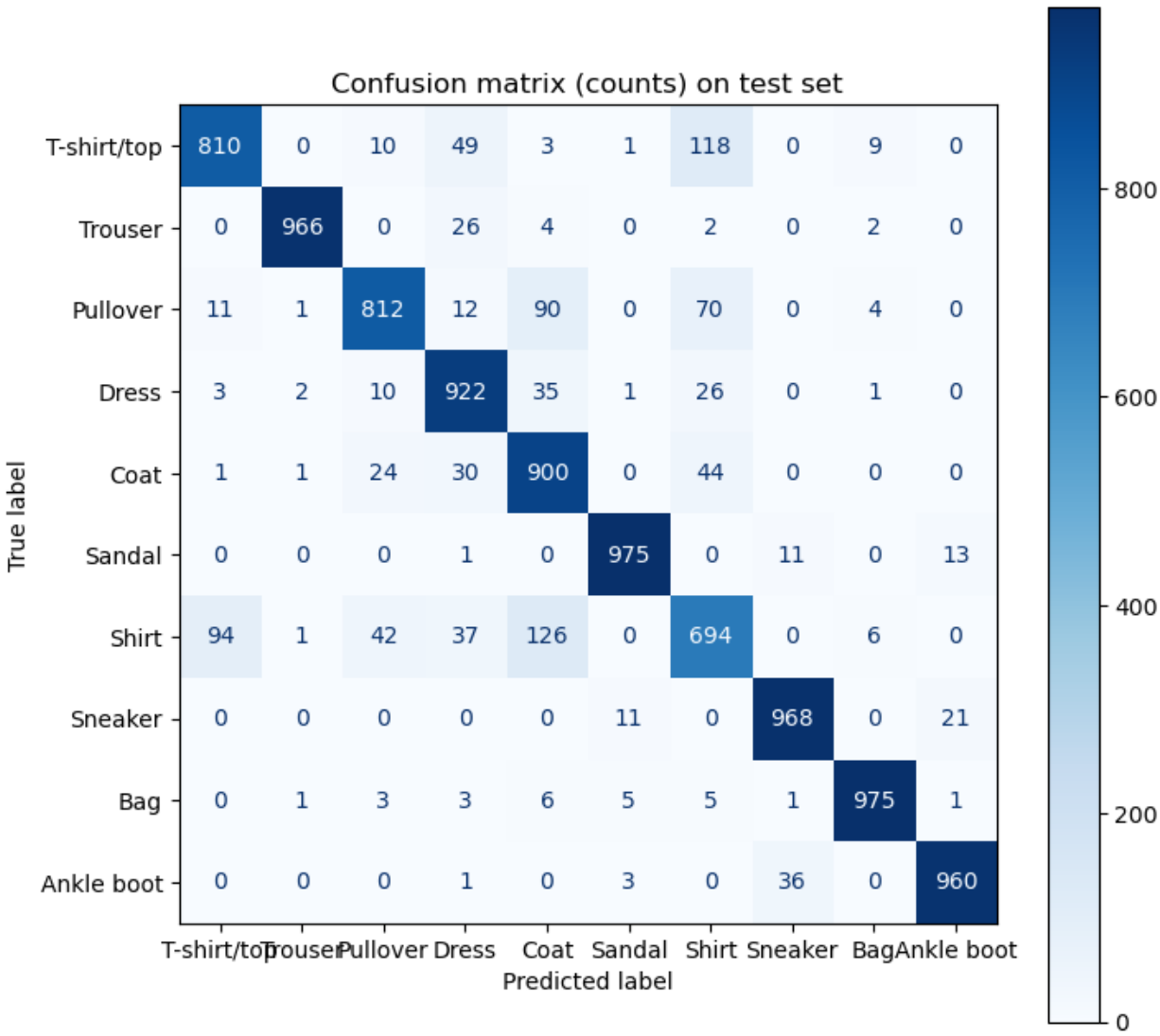


Now evaluate the saved best model on the held-out test set; compute confusion matrix and per-class metrics.

```
test_loss, test_acc, test_preds, test_labels = evaluate(model,
test_loader, criterion, DEVICE)
print(f"Test loss: {test_loss:.4f}  Test acc: {test_acc:.4f}")
print("\nClassification report:\n", classification_report(test_labels,
test_preds, target_names=[labels_map[i] for i in range(10)]))

# Plot confusion matrix (raw counts)
cm = confusion_matrix(test_labels, test_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=[labels_map[i] for i in range(10)])
fig, ax = plt.subplots(figsize=(8,8))
disp.plot(ax=ax, cmap='Blues', colorbar=True)
plt.title("Confusion matrix (counts) on test set")
plt.show()
```

```
Test loss: 0.2858  Test acc: 0.8982

Classification report:
               precision    recall  f1-score   support

 T-shirt/top       0.88      0.81      0.84      1000
     Trouser       0.99      0.97      0.98      1000
```

```
    Pullover       0.90      0.81      0.85      1000
       Dress       0.85      0.92      0.89      1000
        Coat       0.77      0.90      0.83      1000
      Sandal       0.98      0.97      0.98      1000
       Shirt       0.72      0.69      0.71      1000
     Sneaker       0.95      0.97      0.96      1000
         Bag       0.98      0.97      0.98      1000
  Ankle boot       0.96      0.96      0.96      1000

    accuracy                           0.90     10000
   macro avg       0.90      0.90      0.90     10000
weighted avg       0.90      0.90      0.90     10000
```



We now extract embeddings (feature vectors) from a pretrained ResNet18. These embeddings are the "semantic" representations we will later use for retrieval, surrogate modelling, and to demonstrate feature-based k-NN.

Notes:

- ResNet expects 3-channel images of size 224×224 and ImageNet normalization.
- Our Fashion-MNIST images are 28×28 grayscale — we convert to 3-channel by repeating the channel and resize to 224.
- Extraction pipeline:
    a. Build a feature-extractor model that returns the pooled feature vector (512-D for ResNet18).
    b. Run the test_loader through the extractor and store embeddings + labels.

```
from torchvision import models

# Feature extractor: ResNet18 without final FC
resnet = models.resnet18(pretrained=True)
# create extractor that outputs pooled features
feature_extractor = nn.Sequential(*list(resnet.children())[:-1])  #
all layers except final FC
feature_extractor = feature_extractor.to(DEVICE)
feature_extractor.eval()

# transform for ResNet: resize to 224 and convert to 3 channels
resnet_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.Lambda(lambda img: img.repeat(3,1,1) if isinstance(img,
torch.Tensor) and img.shape[0]==1 else img),  # replicate channel
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
])

# Build a DataLoader that supplies images transformed for ResNet (use
test set for demo)
class ResNetFashionDataset(torch.utils.data.Dataset):
    def __init__(self, base_dataset, transform):
        self.base = base_dataset
        self.transform = transform
    def __len__(self):
        return len(self.base)
    def __getitem__(self, idx):
        img, lbl = self.base[idx]  # img is already tensor (C,H,W)
normalized for FM earlier
        # For base_dataset we used fm_transform (normalized for
FashionMNIST), so we must undo that
        # Instead easier: re-load with plain ToTensor for extraction
purpose
        pil_ds = datasets.FashionMNIST(root=data_dir, train=False,
download=False, transform=transforms.ToTensor())
```

```
        img, lbl = pil_ds[idx]
        # img: (1,28,28), make 3-channel
        img = img.repeat(3,1,1)
        img = transforms.functional.resize(img, (224,224))
        # normalize with ImageNet stats
        img = transforms.functional.normalize(img,
mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
        return img, lbl

# Build loader for embeddings extraction (smaller subset for speed in
demo)
emb_batch_size = 128
resnet_test_ds =
ResNetFashionDataset(datasets.FashionMNIST(root=data_dir, train=False,
download=False, transform=transforms.ToTensor()), transform=None)
resnet_loader = DataLoader(resnet_test_ds, batch_size=emb_batch_size,
shuffle=False, num_workers=0)

# Extract embeddings
all_embeddings = []
all_labels = []
with torch.no_grad():
    for imgs, lbls in tqdm(resnet_loader, desc="Extracting
embeddings"):
        imgs = imgs.to(DEVICE)
        feats = feature_extractor(imgs)  # output shape (B, 512, 1,1)
        feats = feats.view(feats.size(0), -1)  # (B,512)
        all_embeddings.append(feats.cpu().numpy())
        all_labels.append(lbls.numpy())

embeddings = np.vstack(all_embeddings)   # (n_samples, 512)
emb_labels = np.concatenate(all_labels)
print("Embeddings shape:", embeddings.shape, "Labels shape:",
emb_labels.shape)
```

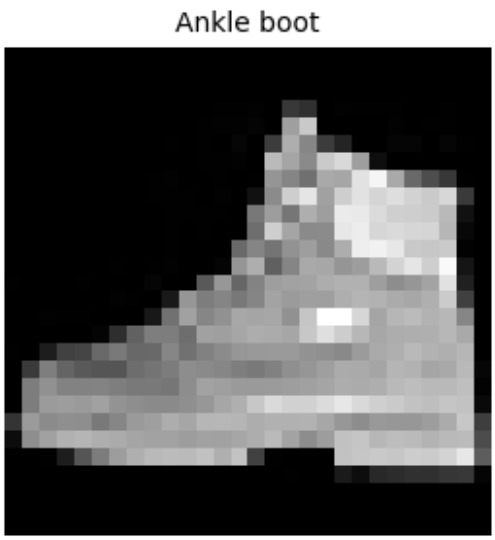
```
C:\Users\hp prodesk\AppData\Roaming\Python\Python312\site-packages\
torchvision\models\_utils.py:208: UserWarning: The parameter
'pretrained' is deprecated since 0.13 and may be removed in the
future, please use 'weights' instead.
  warnings.warn(
C:\Users\hp prodesk\AppData\Roaming\Python\Python312\site-packages\
torchvision\models\_utils.py:223: UserWarning: Arguments other than a
weight enum or `None` for 'weights' are deprecated since 0.13 and may
be removed in the future. The current behavior is equivalent to
passing `weights=ResNet18_Weights.IMAGENET1K_V1`. You can also use
`weights=ResNet18_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
```

```
Downloading: "https://download.pytorch.org/models/resnet18-
f37072fd.pth" to C:\Users\hp prodesk/.cache\torch\hub\checkpoints\
resnet18-f37072fd.pth

100%|██████████| 44.7M/44.7M [00:01<00:00, 29.0MB/s]

{"model_id":"9aff53fc74ff4e9fa849ac42a79c7c8a","version_major":2,"vers
ion_minor":0}

Embeddings shape: (10000, 512) Labels shape: (10000,)
```

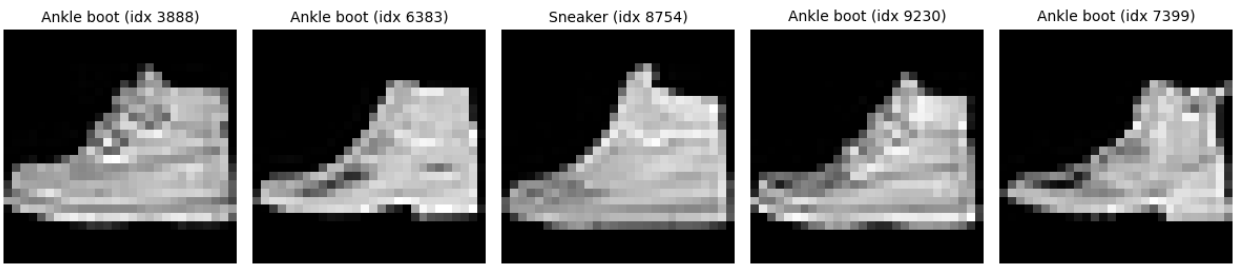
We now demonstrate a very small retrieval: given a query image from the test set, find the top-5 nearest neighbors in embedding space using scikit-learn's `NearestNeighbors`. This is a CPU demonstration — in Notebook 2 we'll use FAISS for faster & larger-scale retrieval.

```
from sklearn.neighbors import NearestNeighbors

# fit k-NN on embeddings
knn = NearestNeighbors(n_neighbors=6, metric='cosine')  # 6 because
first neighbor will be the query itself
knn.fit(embeddings)

# pick a random query index
qry_idx = np.random.randint(0, embeddings.shape[0])
qry_emb = embeddings[qry_idx:qry_idx+1]
dists, idxs = knn.kneighbors(qry_emb, n_neighbors=6)
neighbor_idxs = idxs[0][1:]   # skip the query itself
print("Query index:", qry_idx, " label:",
labels_map[int(emb_labels[qry_idx])])

# load original images for display: using simple ToTensor loader as
earlier
pil_ds = datasets.FashionMNIST(root=data_dir, train=False,
download=False, transform=transforms.ToTensor())
qry_img, qry_lbl = pil_ds[qry_idx]
neighbor_imgs = [pil_ds[i][0].squeeze(0).numpy() for i in
neighbor_idxs]
titles = [f"{labels_map[int(emb_labels[i])]} (idx {i})" for i in
neighbor_idxs]

# Show query + neighbors
print("Query image:")
show_image_grid([qry_img.squeeze(0).numpy()],
titles=[labels_map[int(qry_lbl)]], ncols=1, figsize=(3,3))
print("Top-5 neighbors (by cosine similarity in ResNet embedding
space):")
show_image_grid(neighbor_imgs, titles=titles, ncols=5, figsize=(12,3))
```

```
Query index: 4002  label: Ankle boot
Query image:
```




```
Top-5 neighbors (by cosine similarity in ResNet embedding space):
```




```
# Measure approximate inference time for a single forward pass on
CPU/GPU for the SmallCNN model.
model.eval()
dummy_input = torch.randn(1,1,28,28).to(DEVICE)
# warm-up
for _ in range(5):
    _ = model(dummy_input)

# timed runs
runs = 50
start = time.time()
for _ in range(runs):
    _ = model(dummy_input)
torch.cuda.synchronize() if torch.cuda.is_available() else None
elapsed = time.time() - start
print(f"Avg inference time over {runs} runs: {elapsed / runs *
1000:.3f} ms")
```

```
Avg inference time over 50 runs: 0.940 ms
```

Checkpoint notes:

- We saved checkpoints as `state_dict` inside the training loop; this is the recommended lightweight approach.
- To load a saved checkpoint:
- For deployment, consider exporting a traced `TorchScript` or `ONNX` model. This notebook intentionally uses `state_dict` saves because they are readable & editable by instructors/students.

Embedding notes:

- Pretrained ResNet embeddings (512-d) capture semantic similarity useful for retrieval and surrogate modeling.
- For downstream surrogate models, you can train a small MLP on these embeddings (fast) instead of fine-tuning the whole ResNet (slow).

Final Exercises (Chunk 5) & Mini-project ideas

1. **Train longer & analyze**: Increase `num_epochs` to 20 and observe learning curves. Report final test accuracy and whether the model overfits.
2. **Weight decay & learning rate**: Re-run training with `weight_decay=1e-4` in Adam and compare validation performance.
3. **Embedding-based classifier (fast surrogate)**:
   - Using the ResNet embeddings you extracted for the training set, train a simple LogisticRegression or MLP classifier on those embeddings (scikit-learn or PyTorch).
   - Compare test accuracy and inference latency to the SmallCNN and discuss tradeoffs (accuracy vs speed).
4. **k-NN retrieval evaluation**:
   - Compute precision@k (k=1,3,5) using label agreement as ground-truth; report values for the ResNet embeddings.
5. **Model export (optional)**:
   - Export SmallCNN to TorchScript: `torch.jit.trace(model, example_input)` and run a sample inference from the traced model.
6. **Write-up**: For each experiment, include a 1-paragraph interpretation of results and list possible next steps.

Mini-project:

- Combine surrogate training & retrieval: precompute embeddings for a chosen MedMNIST dataset, train an MLP surrogate on embeddings, create a small retrieval UI (Colab-friendly display) that given a query shows top-k similar cases & predicts the label via the surrogate. Provide evaluation (AUC, precision@k) and an ethics paragraph on dataset biases and clinical caution.

Notebook summary & next steps

We have completed:

- A practical end-to-end pipeline: data loading → small CNN → training loop → checkpointing → evaluation.
- Techniques to extract embeddings from pretrained models and a quick retrieval demo.
- Tools and habits for debugging, reproducibility, and basic model export.

Next: **Notebook 2** will build on these skills: we'll use Hugging Face Vision Transformers, FAISS for large-scale retrieval, fine-tuning strategies, surrogate modelling (distillation & embedding→MLP), model export (ONNX/TorchScript) and evaluation using medically-relevant metrics (AUC, sensitivity, specificity) with MedMNIST.

```
pip install pandoc

Collecting pandoc
  Downloading pandoc-2.4.tar.gz (34 kB)
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'done'
Collecting plumbum (from pandoc)
  Downloading plumbum-1.10.0-py3-none-any.whl.metadata (8.4 kB)
Requirement already satisfied: ply in c:\users\priya\anaconda3\lib\
site-packages (from pandoc) (3.11)
Requirement already satisfied: pywin32 in c:\users\priya\anaconda3\
lib\site-packages (from plumbum->pandoc) (305.1)
Downloading plumbum-1.10.0-py3-none-any.whl (127 kB)
Building wheels for collected packages: pandoc
  Building wheel for pandoc (setup.py): started
  Building wheel for pandoc (setup.py): finished with status 'done'
  Created wheel for pandoc: filename=pandoc-2.4-py3-none-any.whl
size=34821
sha256=db6fc98efe3a7e33ed9ebf65d7737b3f1568a6389128006b975f11f26d9c163
2
  Stored in directory: c:\users\priya\appdata\local\pip\cache\wheels\
9c\2f\9f\b1aac8c3e74b4ee327dc8c6eac5128996f9eadf586e2c0ba67
Successfully built pandoc
Installing collected packages: plumbum, pandoc
Successfully installed pandoc-2.4 plumbum-1.10.0
Note: you may need to restart the kernel to use updated packages.

pip install nbconvert

Requirement already satisfied: nbconvert in c:\users\priya\anaconda3\
lib\site-packages (7.16.4)
Requirement already satisfied: beautifulsoup4 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (4.12.3)
Requirement already satisfied: bleach!=5.0.0 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (4.1.0)
Requirement already satisfied: defusedxml in c:\users\priya\anaconda3\
lib\site-packages (from nbconvert) (0.7.1)
Requirement already satisfied: jinja2>=3.0 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (3.1.4)
Requirement already satisfied: jupyter-core>=4.7 in c:\users\priya\
```

```
anaconda3\lib\site-packages (from nbconvert) (5.7.2)
Requirement already satisfied: jupyterlab-pygments in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (0.1.2)
Requirement already satisfied: markupsafe>=2.0 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (2.1.3)
Requirement already satisfied: mistune<4,>=2.0.3 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (2.0.4)
Requirement already satisfied: nbclient>=0.5.0 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (0.8.0)
Requirement already satisfied: nbformat>=5.7 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (5.10.4)
Requirement already satisfied: packaging in c:\users\priya\anaconda3\
lib\site-packages (from nbconvert) (24.1)
Requirement already satisfied: pandocfilters>=1.4.1 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (1.5.0)
Requirement already satisfied: pygments>=2.4.1 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (2.15.1)
Requirement already satisfied: tinycss2 in c:\users\priya\anaconda3\
lib\site-packages (from nbconvert) (1.2.1)
Requirement already satisfied: traitlets>=5.1 in c:\users\priya\
anaconda3\lib\site-packages (from nbconvert) (5.14.3)
Requirement already satisfied: six>=1.9.0 in c:\users\priya\anaconda3\
lib\site-packages (from bleach!=5.0.0->nbconvert) (1.16.0)
Requirement already satisfied: webencodings in c:\users\priya\
anaconda3\lib\site-packages (from bleach!=5.0.0->nbconvert) (0.5.1)
Requirement already satisfied: platformdirs>=2.5 in c:\users\priya\
anaconda3\lib\site-packages (from jupyter-core>=4.7->nbconvert)
(3.10.0)
Requirement already satisfied: pywin32>=300 in c:\users\priya\
anaconda3\lib\site-packages (from jupyter-core>=4.7->nbconvert)
(305.1)
Requirement already satisfied: jupyter-client>=6.1.12 in c:\users\
priya\anaconda3\lib\site-packages (from nbclient>=0.5.0->nbconvert)
(8.6.0)
Requirement already satisfied: fastjsonschema>=2.15 in c:\users\priya\
anaconda3\lib\site-packages (from nbformat>=5.7->nbconvert) (2.16.2)
Requirement already satisfied: jsonschema>=2.6 in c:\users\priya\
anaconda3\lib\site-packages (from nbformat>=5.7->nbconvert) (4.23.0)
Requirement already satisfied: soupsieve>1.2 in c:\users\priya\
anaconda3\lib\site-packages (from beautifulsoup4->nbconvert) (2.5)
Requirement already satisfied: attrs>=22.2.0 in c:\users\priya\
anaconda3\lib\site-packages (from jsonschema>=2.6->nbformat>=5.7-
>nbconvert) (23.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
c:\users\priya\anaconda3\lib\site-packages (from jsonschema>=2.6-
>nbformat>=5.7->nbconvert) (2023.7.1)
Requirement already satisfied: referencing>=0.28.4 in c:\users\priya\
anaconda3\lib\site-packages (from jsonschema>=2.6->nbformat>=5.7-
>nbconvert) (0.30.2)
```

```
Requirement already satisfied: rpds-py>=0.7.1 in c:\users\priya\
anaconda3\lib\site-packages (from jsonschema>=2.6->nbformat>=5.7-
>nbconvert) (0.10.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\
priya\anaconda3\lib\site-packages (from jupyter-client>=6.1.12-
>nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: pyzmq>=23.0 in c:\users\priya\
anaconda3\lib\site-packages (from jupyter-client>=6.1.12-
>nbclient>=0.5.0->nbconvert) (25.1.2)
Requirement already satisfied: tornado>=6.2 in c:\users\priya\
anaconda3\lib\site-packages (from jupyter-client>=6.1.12-
>nbclient>=0.5.0->nbconvert) (6.4.1)
Note: you may need to restart the kernel to use updated packages.
```

```
  jupyter nbconvert --to pdf your_notebook.ipynb
```

```
  Cell In[8], line 1
    jupyter nbconvert --to pdf your_notebook.ipynb
            ^
SyntaxError: invalid syntax
```